Design Document

for

KaamSetu

- Domestic Work Marketplace

Version 1.0

Prepared by

Group #: 15 **Group Name:** ***Null Pointers***

Ashish Gautam	240210	ashishu24@iitk.ac.in
Chaniyara Yug	240299	chaniyaray24@iitk.ac.in
Harsh Vijay	240436	harshvijay24@iitk.ac.in
Harshit Saini	240442	harshits24@iitk.ac.in
Himanshu Yadav	240455	himanshuy24@iitk.ac.in
Ninad Tagade	240699	ninadt24@iitk.ac.in
Nirajkumar Binod Prasad	240701	nirajkp24@iitk.ac.in
Piyush Santosh Ghayal	240748	santosh24@iitk.ac.in
Vaidik	241130	vaidik24@iitk.ac.in
Virag Rathod	241170	virag24@iitk.ac.in

Course: CS253

Mentor TA: *Utkarsh Agrawal*

Date: 06 / 02 / 2026

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Version 1.0	Ashish Gautam Chaniyara Yug Harsh Vijay Harshit Saini Himanshu Yadav Ninad Tagade Nirajkumar Binod Prasad Piyush Santosh Ghayal Vaidik Virag Rathod	Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded.	06/02/2026

1 Context Design

1.1 Context Model

The system works smoothly with its different subsystems, all of which come together to create the best possible service marketplace experience for users. These subsystems are linked to each other and the server, handling necessary fetch and update requests. Often, using one subsystem naturally leads to using another as a follow-up.

The model and its subsystems are as follows:

- **User Authentication System:** Using the user's email or phone number, the system authenticates the signup for first-time login using OTP, ensuring secure access for both service seekers and workers.
- **User Profile Management:** The system maintains the user profiles of the **service seekers (employers)** and **domestic workers** using databases and the various associated attributes while allowing them to add or remove several features (e.g., updating address or skills).
- **Job Request & Posting System:** The service seeker is given the privilege of posting a job requirement (e.g., "Plumber needed" or "Cook hired"), where they can provide details about the task, including but not limited to timings, specific work required, and location.
- **Rating and Review System:** One of the system's key features, it allows the employers to review and rate the workers they have hired as well as the quality of service provided. The workers also have the option to do the same for any employer, hence providing a two-fold review system, ensuring impartiality for both sides.
- **Searching and Filtering System:** The system provides a simple yet efficient search and filtering system, allowing users to search for **"Live Jobs"** or specific worker categories (e.g., Maid, Cook, Electrician) according to their preferences/choices. This eases their process of finding the right help.
- **Messaging System:** Allows users to connect easily with each other if they are interested in a posted job or a worker's profile. One can easily chat over any concerns before visiting the location or finalizing the hiring.
- **Referral System:** A list of trusted workers that a user has previously employed and vouched for. This allows users to share contacts of reliable workers (e.g., "Amit Sharma" referring a Plumber) with their network, building a community of trust.
- **Preference-based Matching System:** The searching filter allows users to set their preferences on essential questions (e.g., part-time vs. full-time, specific skills) and then provides them with the best possible matches. This will enable them to navigate and make an efficient hiring choice easily.

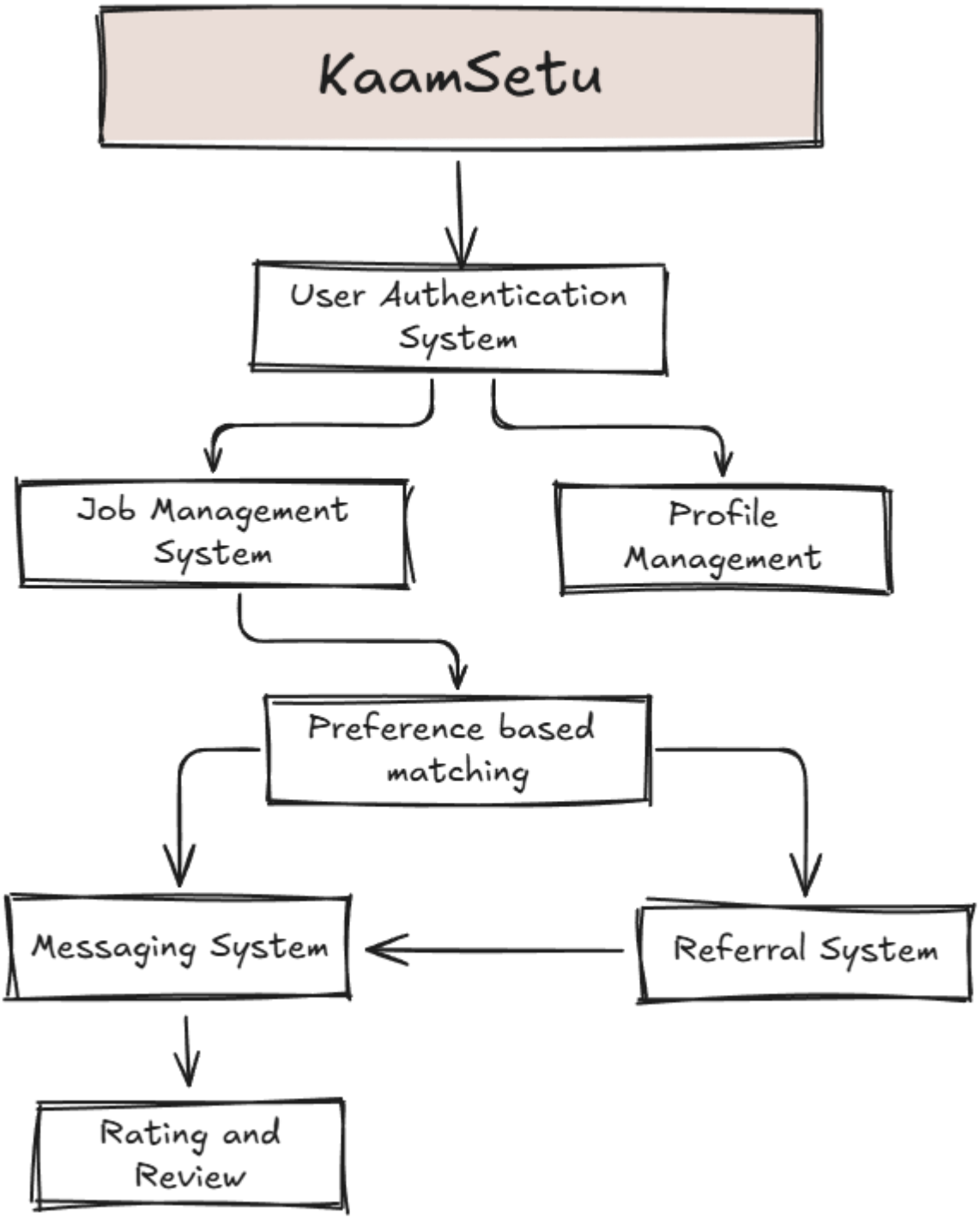


Figure 1: Context Model Diagram

1.2 Human Interface Design

This section includes early drafts that lay out the interfaces for different software parts. These drafts act as a handy guide for developers and system designers, giving a clear breakdown of how the software is structured and how users will interact with it. They cover key features, provide visual examples of screen layouts and outline the different options available to users. The goal is to make everything feel smooth and intuitive, ensuring a user-friendly experience. Using these drafts, developers can keep things consistent and well-organized throughout development.

1.2.1 Authentication & Onboarding Design

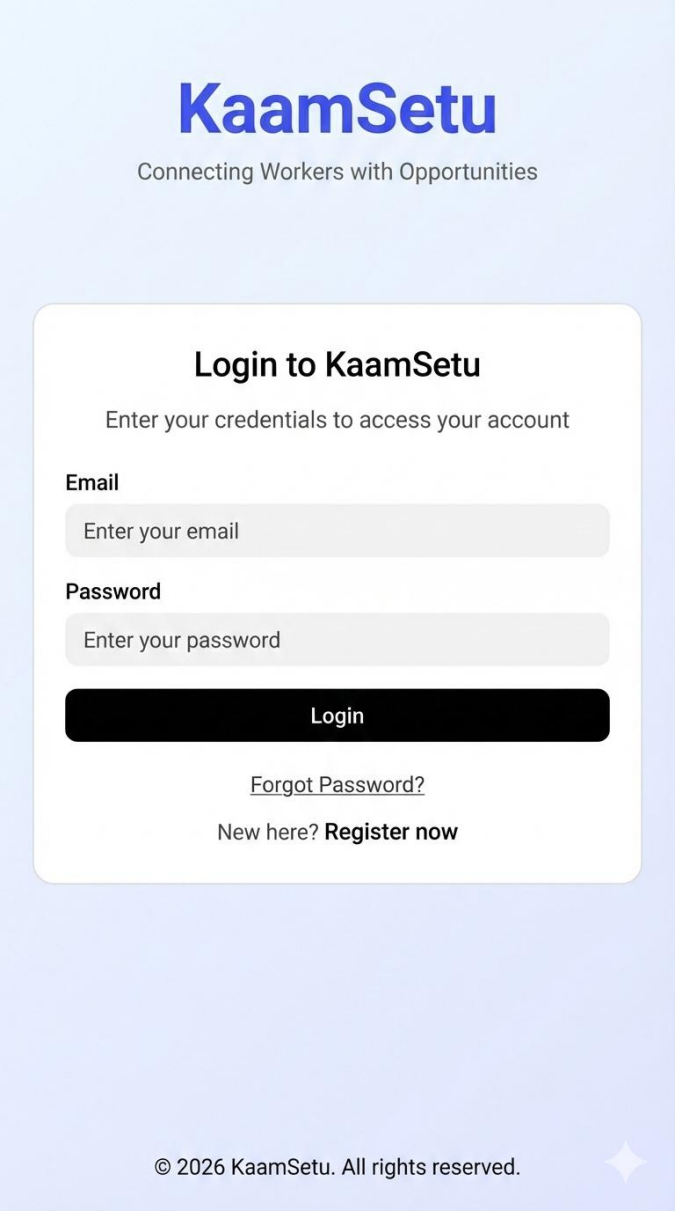


Figure 2: Login Page (For Both User and Worker Login)

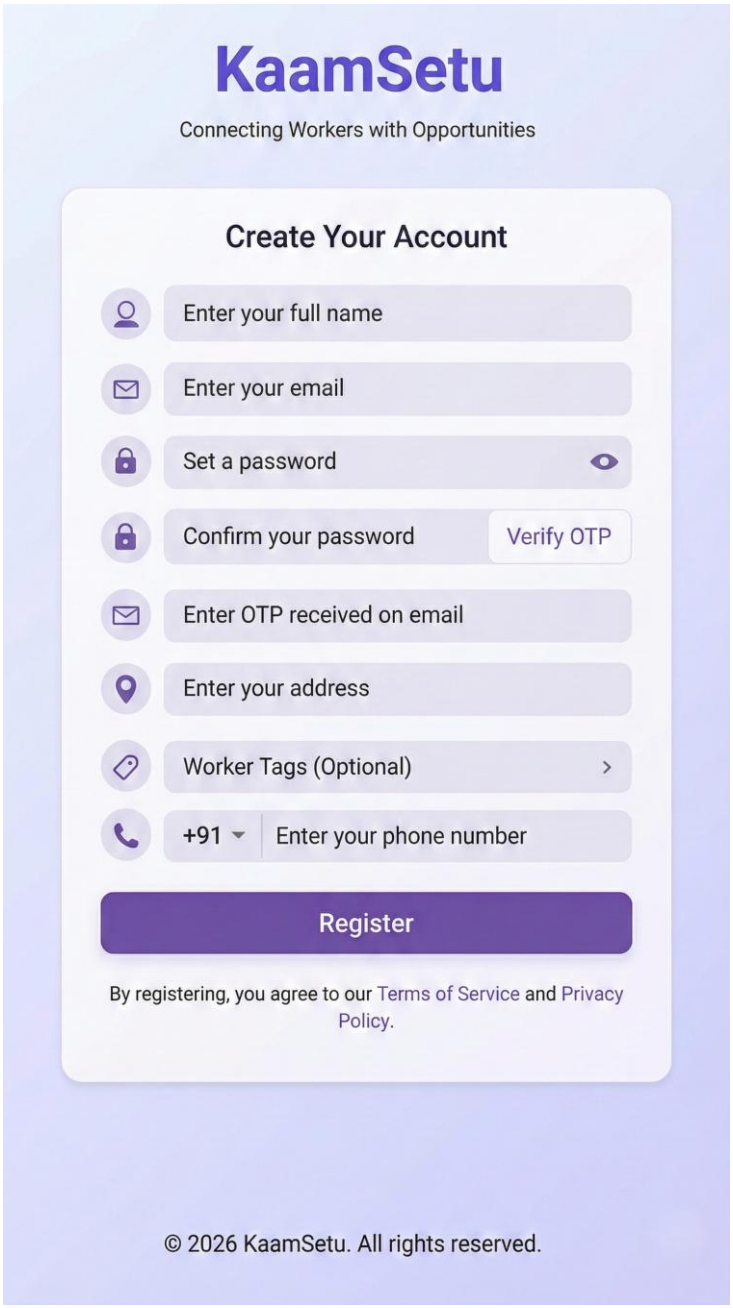


Figure 3: New registration page capturing essential user details including full name, contact information, address, and optional worker skill tags for dual role profiles.

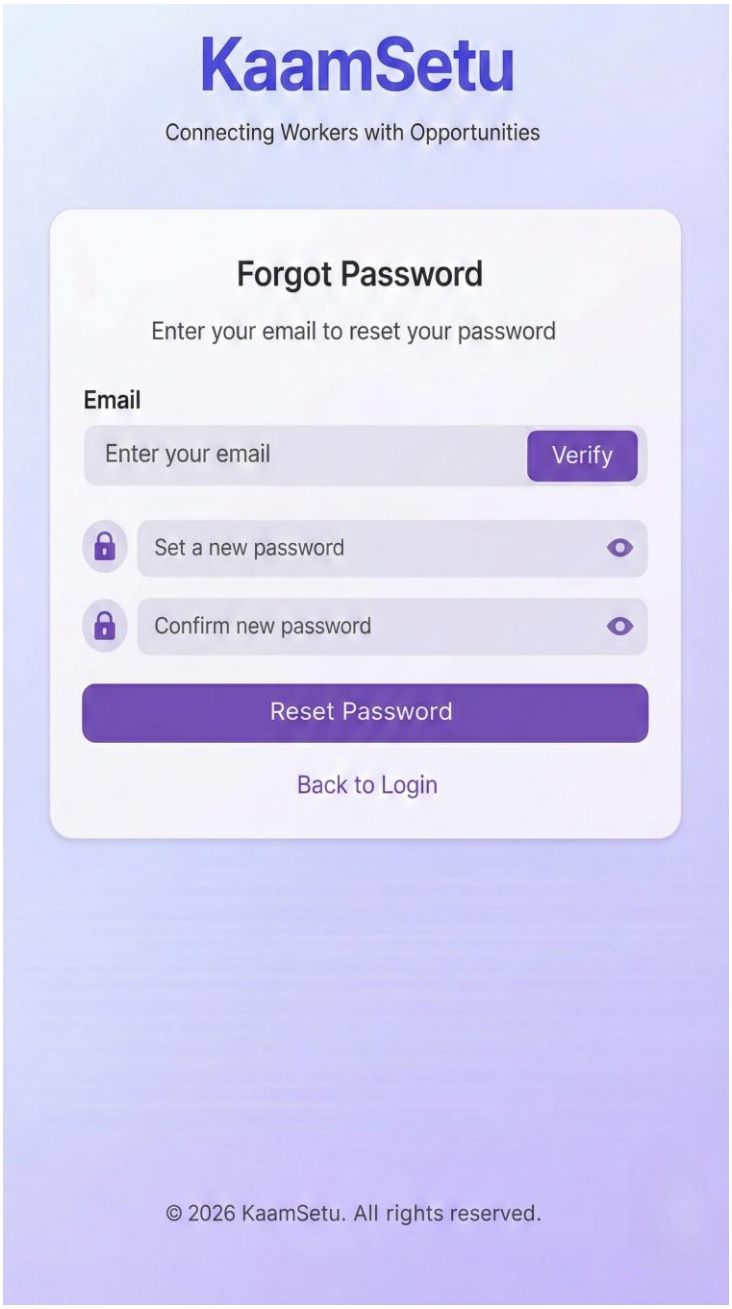


Figure 4: Forgot Password & Account Recovery Screen.

1.2.2 Work Request Posting Platform Design

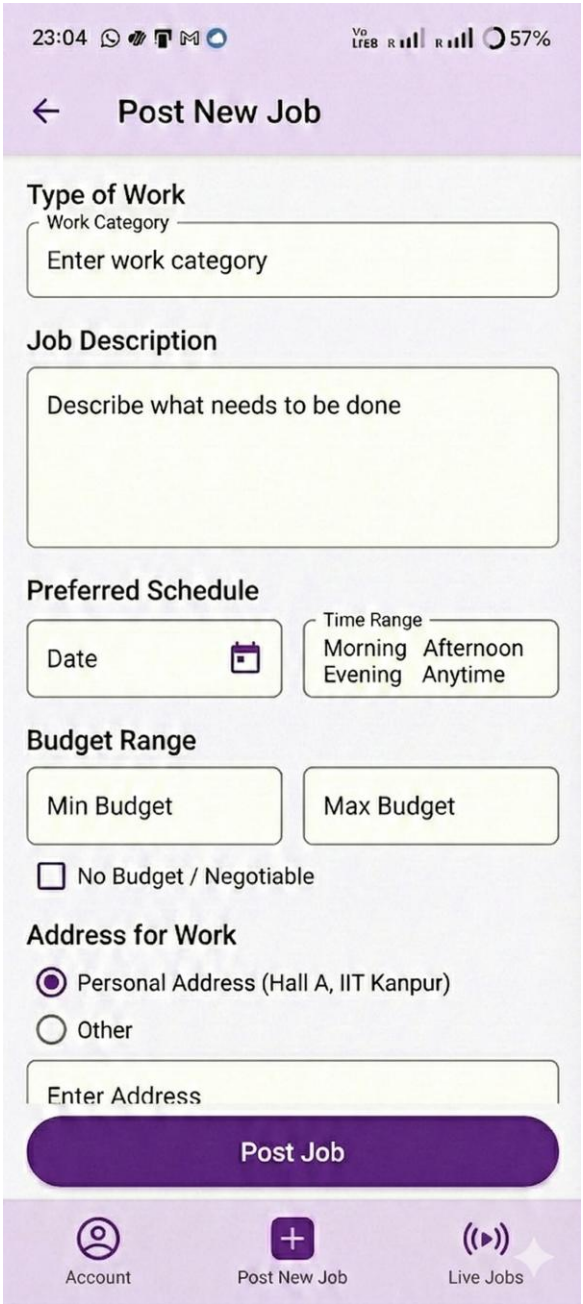


Figure 5: User can create New Job Post/Request, Providing details such as Job Description, Budget, Schedule.

1.2.3 Work Searching & Application Platform Design

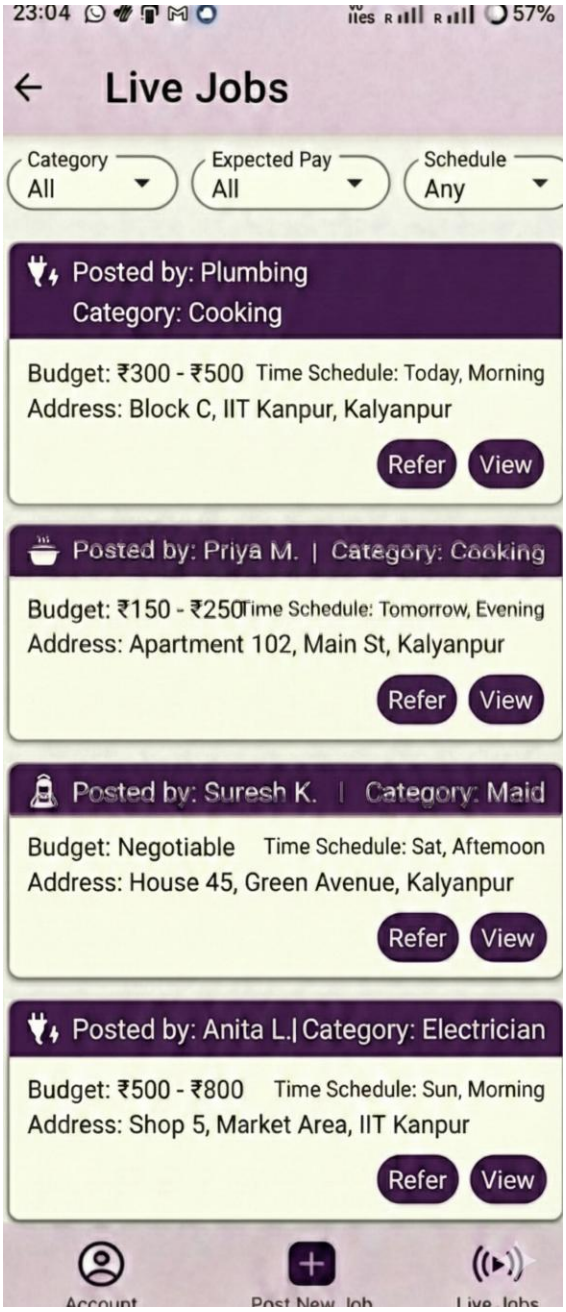


Figure 6: User (who is not a worker) can only have a look at live jobs and details plus, can refer a worker (who is not registered on Kaam Setu) for the job.

Figure 7: Worker can look at Current job requests.

Figure 8: Worker can sort the live jobs suited as per his/her domain. Also, they can sort jobs according or Expected pay.

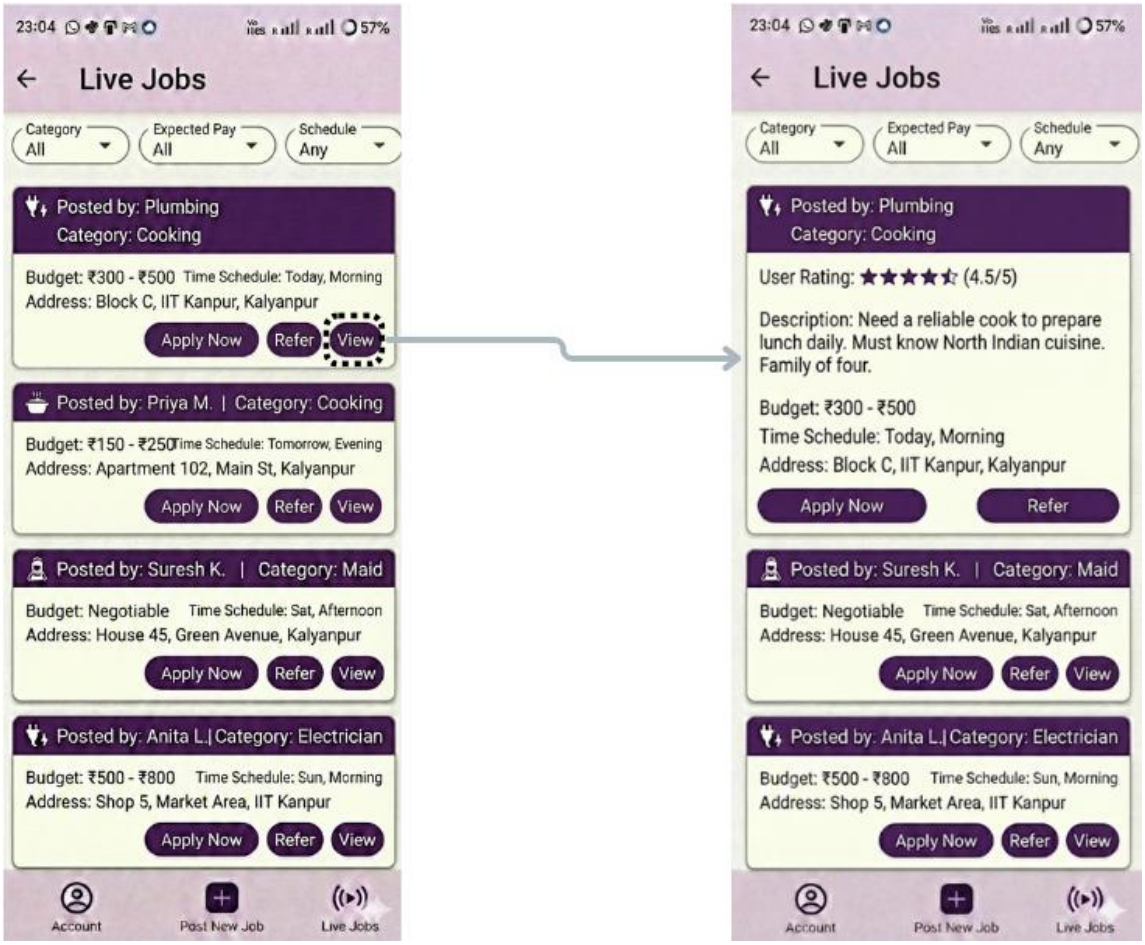


Figure 9: By accessing “View” worker can have a look at Rating of User who posted job and other job details, and worker can select “Apply now” which will create a request for job, it will be reflected in “Accounts” page.

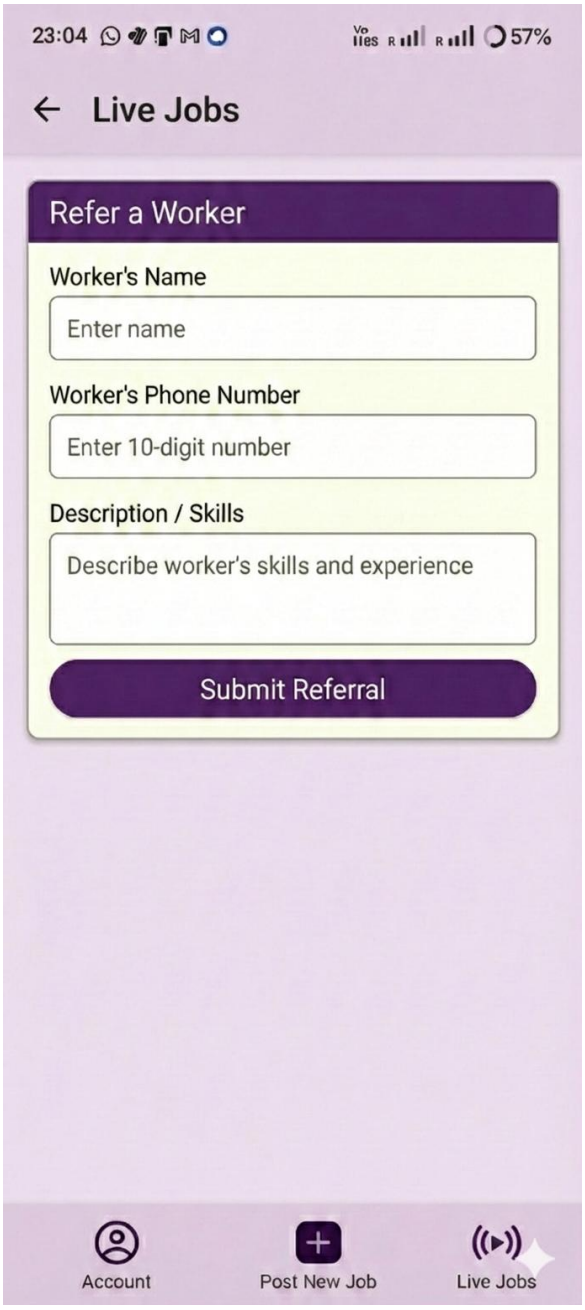


Figure 10: A Worker or User can refer any other worker (who is not registered on Kaam Setu).

1.2.4 Dashboard & Profile Page Design

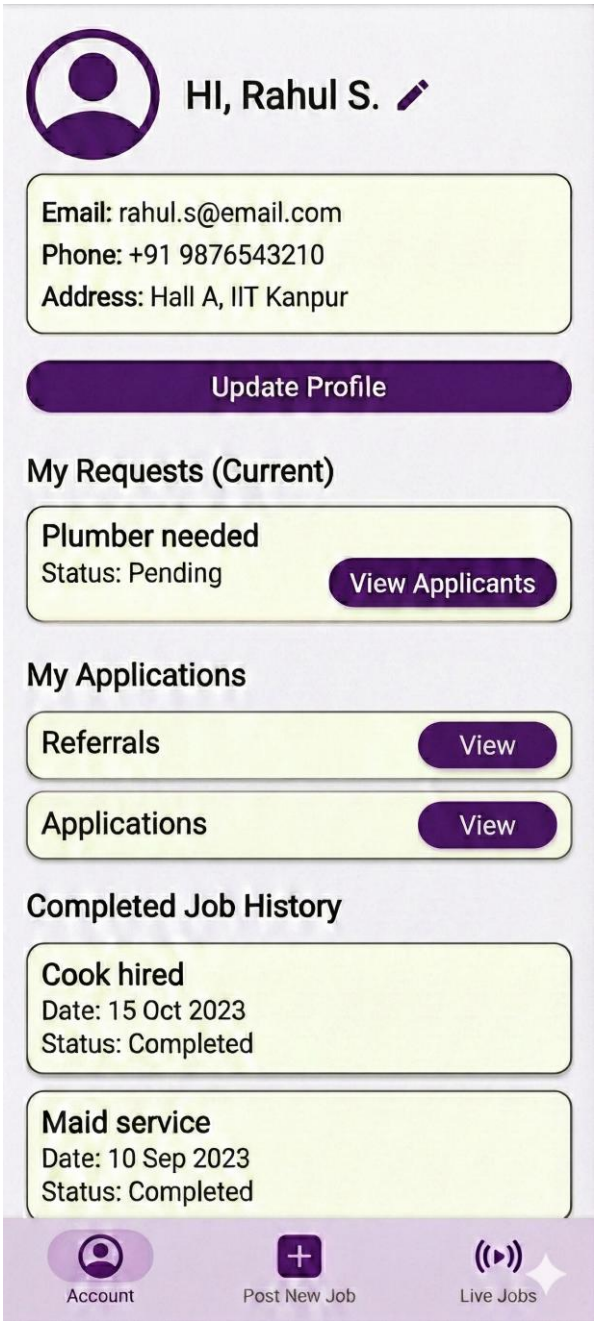


Figure 11: This is the Dashboard the “Account” page, it contains details of user/worker.

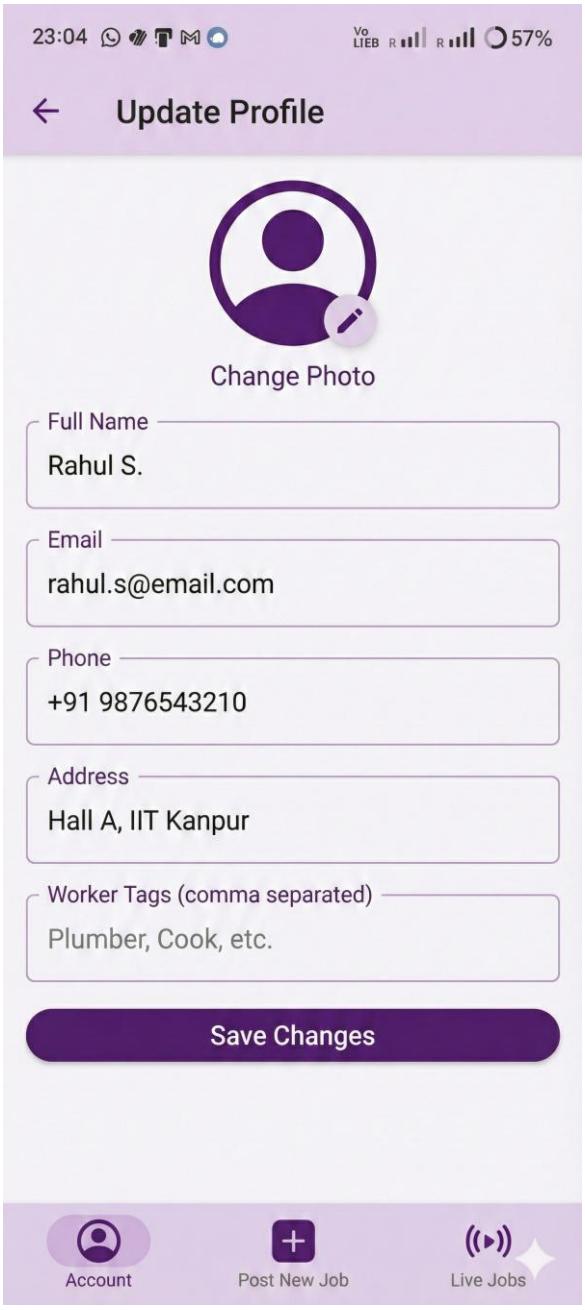


Figure 12: User/Worker can Update profile, Only the user with “Worker Tag” will be considered as a Worker and will be allowed to apply for a job.

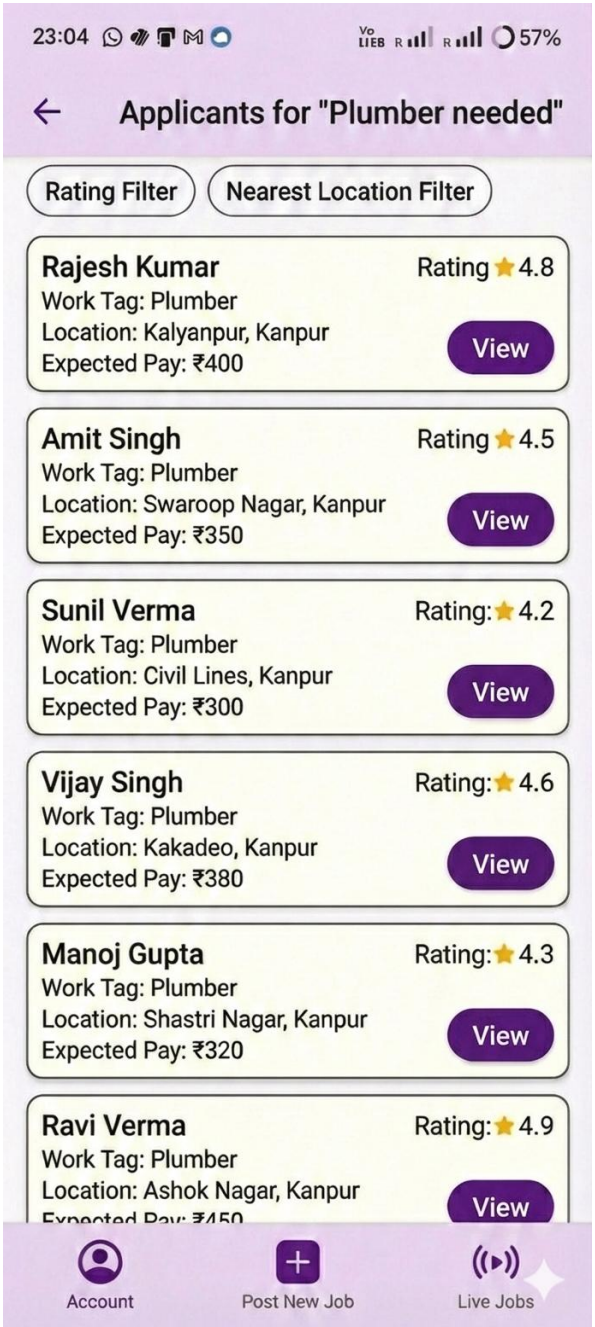


Figure 13: My request on Account page will show this list of Applicants for the Job user posted. User can sort the Request based on Rating and Nearest current location of worker.

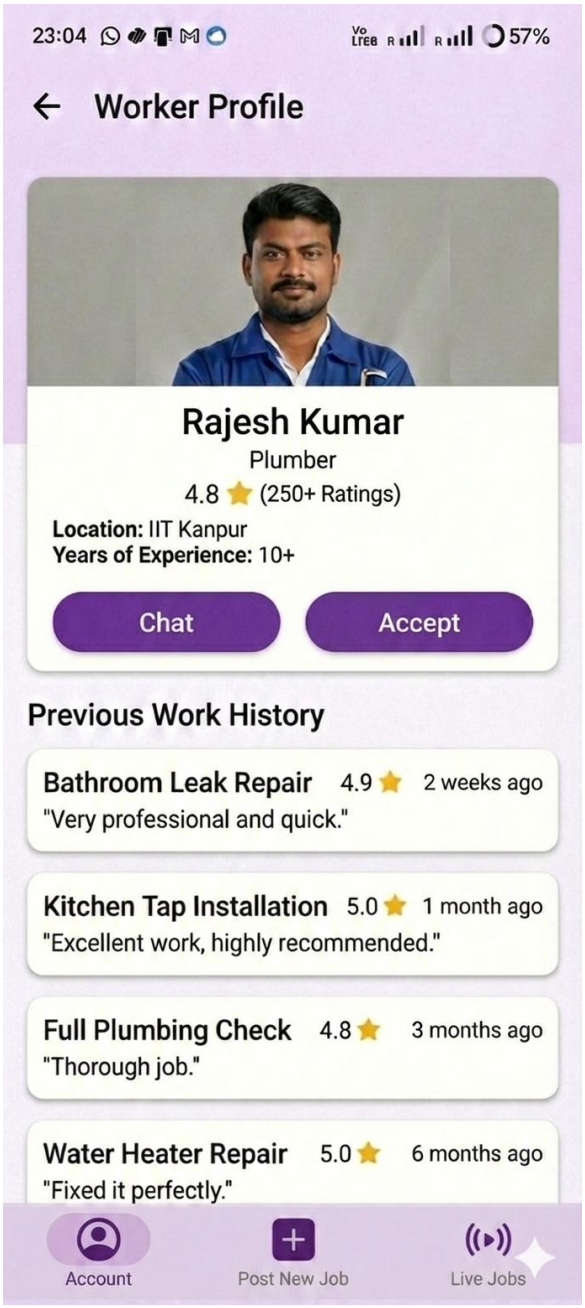


Figure 14: User can view each request, which will consist of Worker Profile and his/her previous work history.

Figure 15: User can have temporary chat with worker.

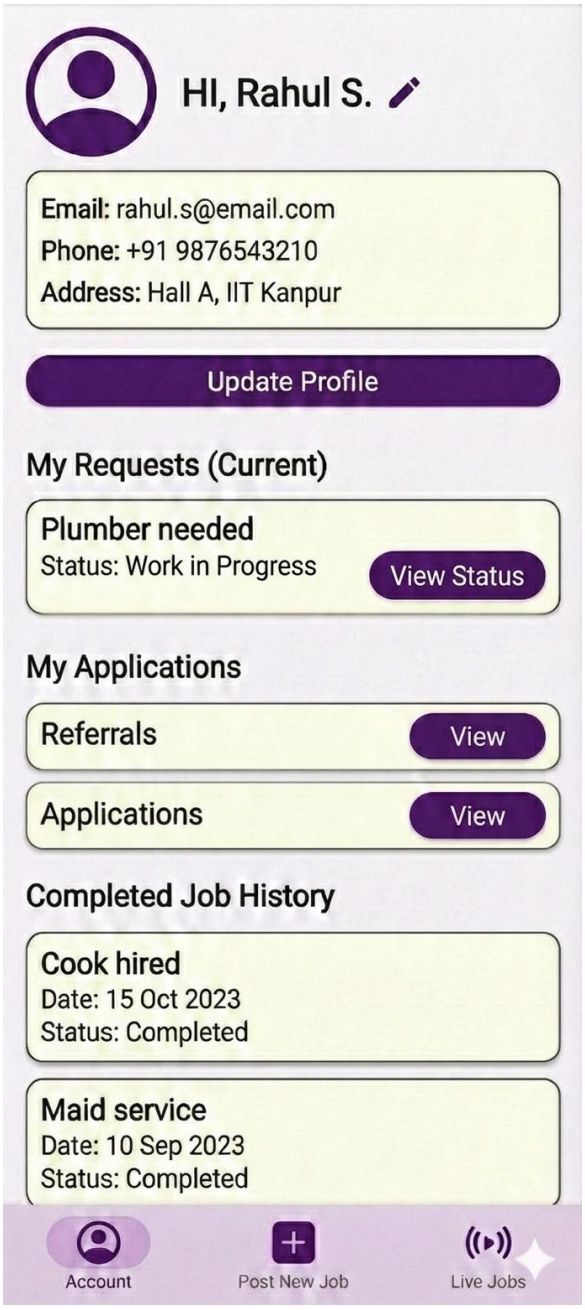


Figure 16: If user Accepts worker request, rest all request will be rejected and this change will reflect in Account Page, “view applicants” is changed to “view status”.

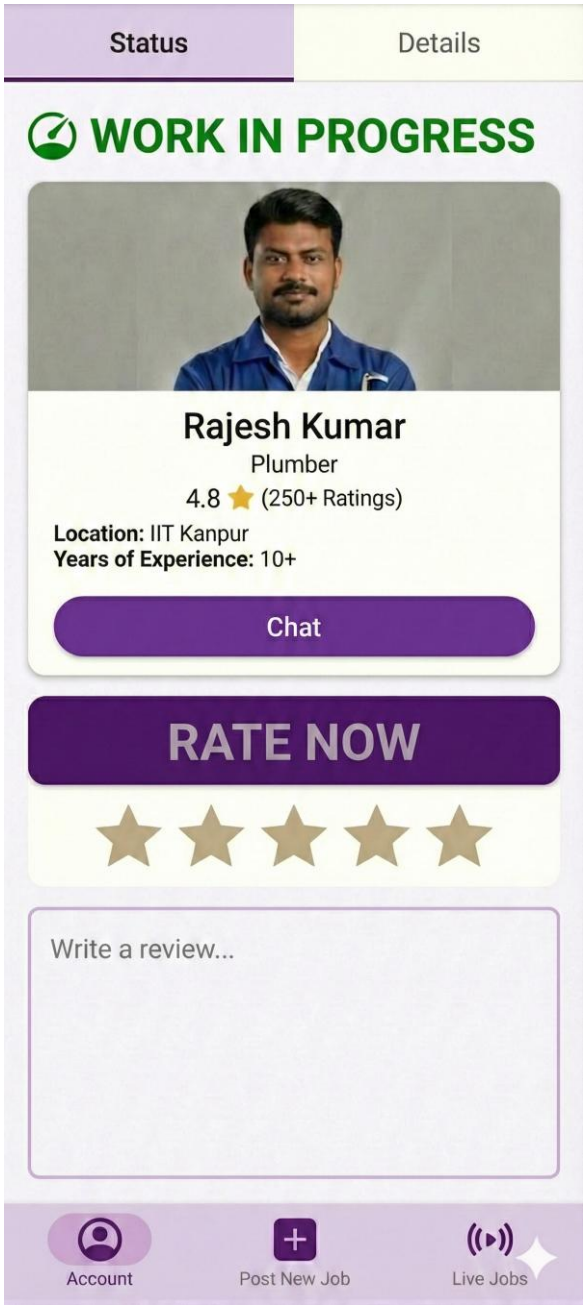


Figure 17: View Status will Show status of work, as well user can Rate worker and give a review when the work begins.

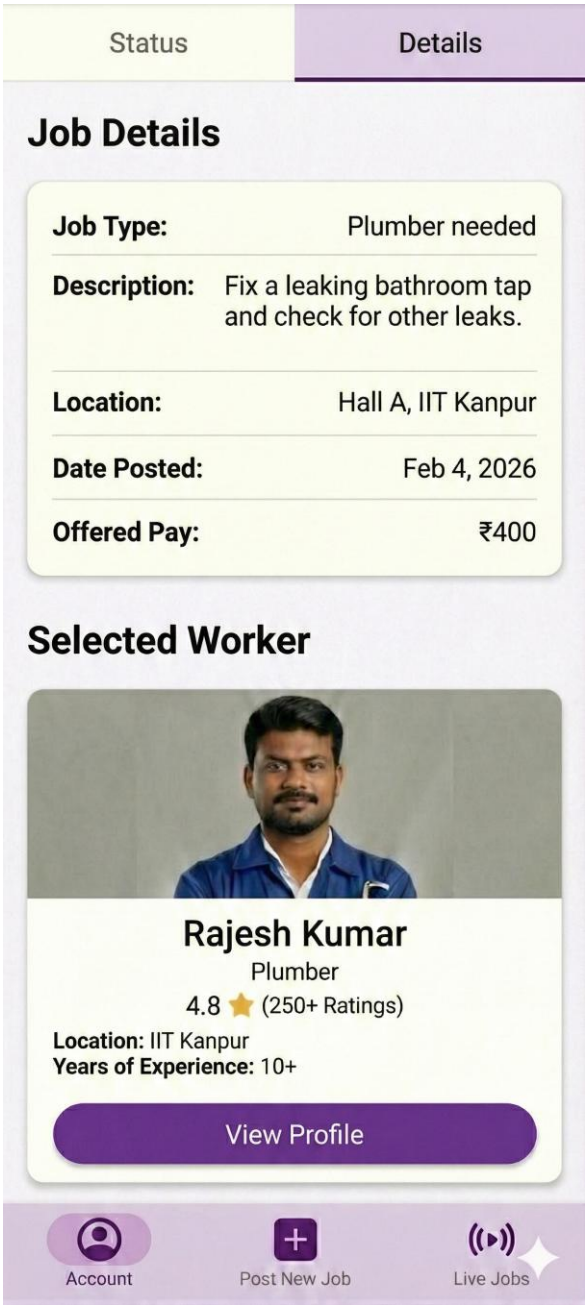


Figure 18: Beside status, Details would also be mentioned about the current work.

Figure 19: Account page consists of “Referrals” which will show list of workers (Unregistered workers) that user/worker have previously referred.

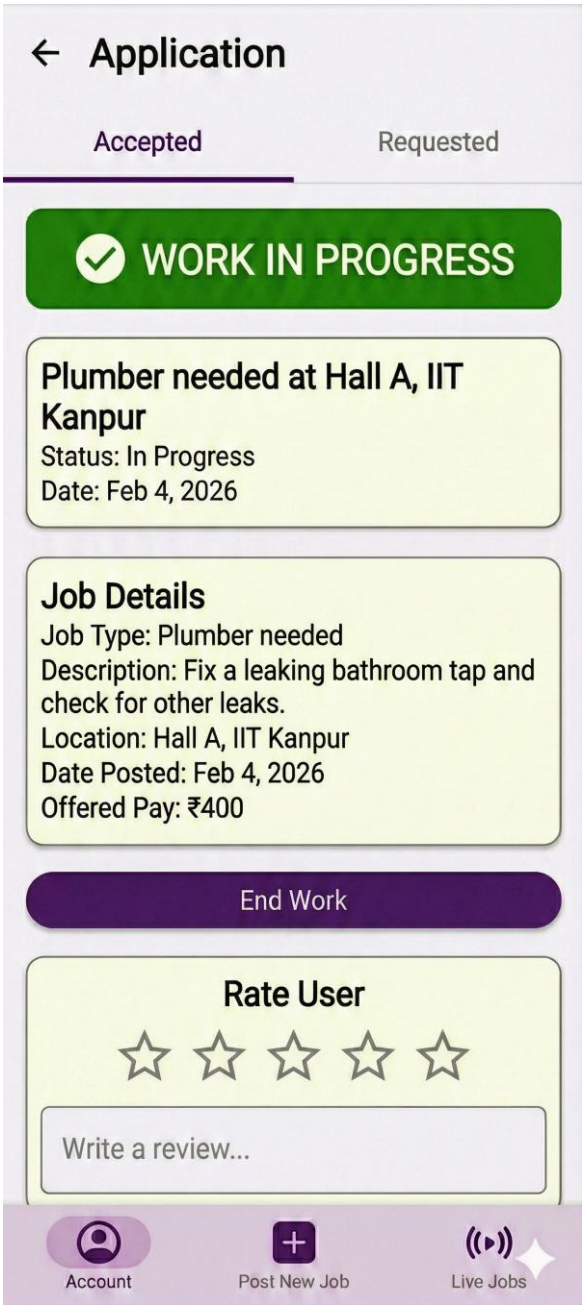


Figure 20: “Application” on the Account Page will be for worker specifically, where he/she can look at jobs which are Accepted, it will contain Job details as well Rate User and review.

Figure 21: In “Application”, requests status for the rest jobs (rest job applications of worker) will also be present.

2 Architecture Design

The architecture of KaamSetu is organized using the **Model–View–Controller (MVC)** design pattern. This architectural approach enables a clear separation of concerns by dividing the system into three distinct components responsible for presentation, control, and data management. The MVC structure simplifies system maintenance, improves modularity, and allows independent evolution of different system components.

The **View** component represents the user-facing interface of the system. It is responsible for displaying information related to user profiles, job postings, job applications, search results, conversations, job history, and ratings. The View provides a dynamic and interactive interface that captures user actions such as submitting forms, initiating searches, applying to jobs, or sending messages. All user-generated requests are forwarded to the Controller for processing.

The **Controller** serves as the central coordinating component of the architecture. It receives requests from the View, validates user input, and manages the overall flow of application logic. The Controller is responsible for handling core functionalities such as authentication and user validation, job request creation and management, application and referral workflows, chat and communication control, and review and rating management. It also manages API interactions and generates appropriate queries required to process user requests.

The **Model** component manages the system's data and core domain logic. It is responsible for maintaining persistent entities such as users, job requests, conversations, and job history. The Model retrieves and updates data in the database based on requests received from the Controller. It also applies filtering and matching logic to support functionalities such as job discovery and worker selection. After processing, the Model supplies structured and relevant data back to the Controller for further handling.

The interaction between the MVC components follows a well-defined flow. User actions are initiated through the View and passed to the Controller. The Controller interprets these actions and interacts with the Model to retrieve or modify data as required. Once processing is complete, the Controller sends the appropriate results back to the View, which updates the user interface accordingly. This structured flow ensures controlled data access, consistent behaviour, and a clear separation between presentation and business logic.

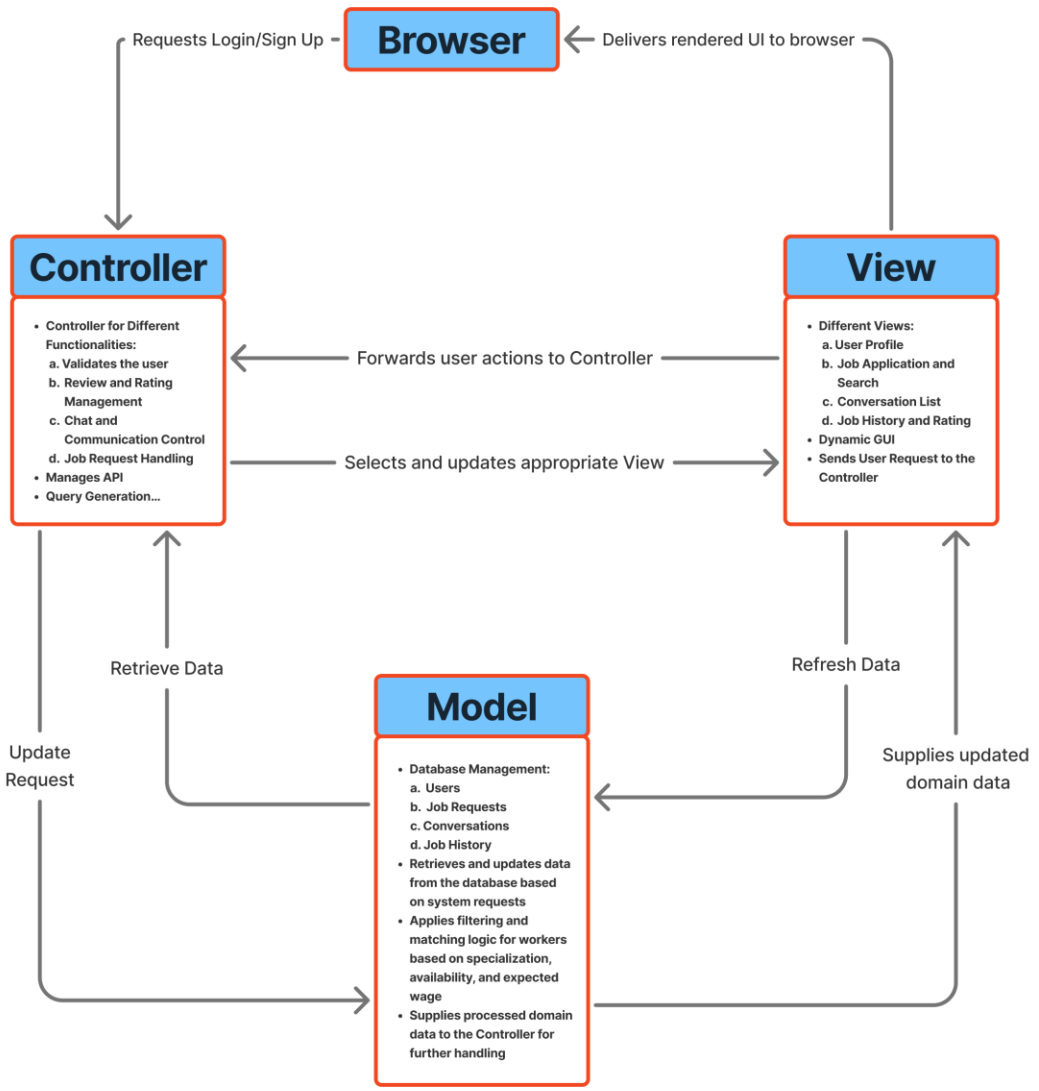


Figure 22: MVC Architecture Design for KaamSetu

3 Object Oriented Design

3.1 Use Case Diagrams

3.1.1 Use Case 1: User and Worker Registration

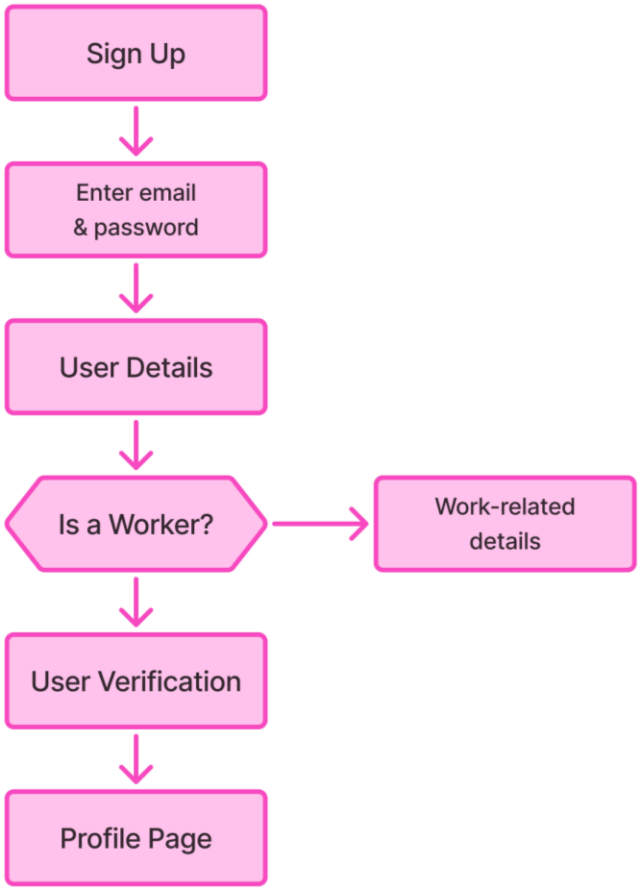


Figure 23: Use Case 1 Flowchart

This use case describes the process through which a new individual creates an account on the KaamSetu platform and establishes their role within the system. A user registers by providing a valid email address, completing OTP-based verification, and setting up authentication credentials. During registration, the user may optionally choose to register as a worker by providing additional work-related details such as service category, experience, and service area. This use case is foundational to the platform, as successful registration and authentication are mandatory prerequisites for accessing all other system functionalities. Upon completion, a verified user account is created and stored in the system database, with an associated worker profile if applicable.

3.1.2 Use Case 2: Posting a Work Request

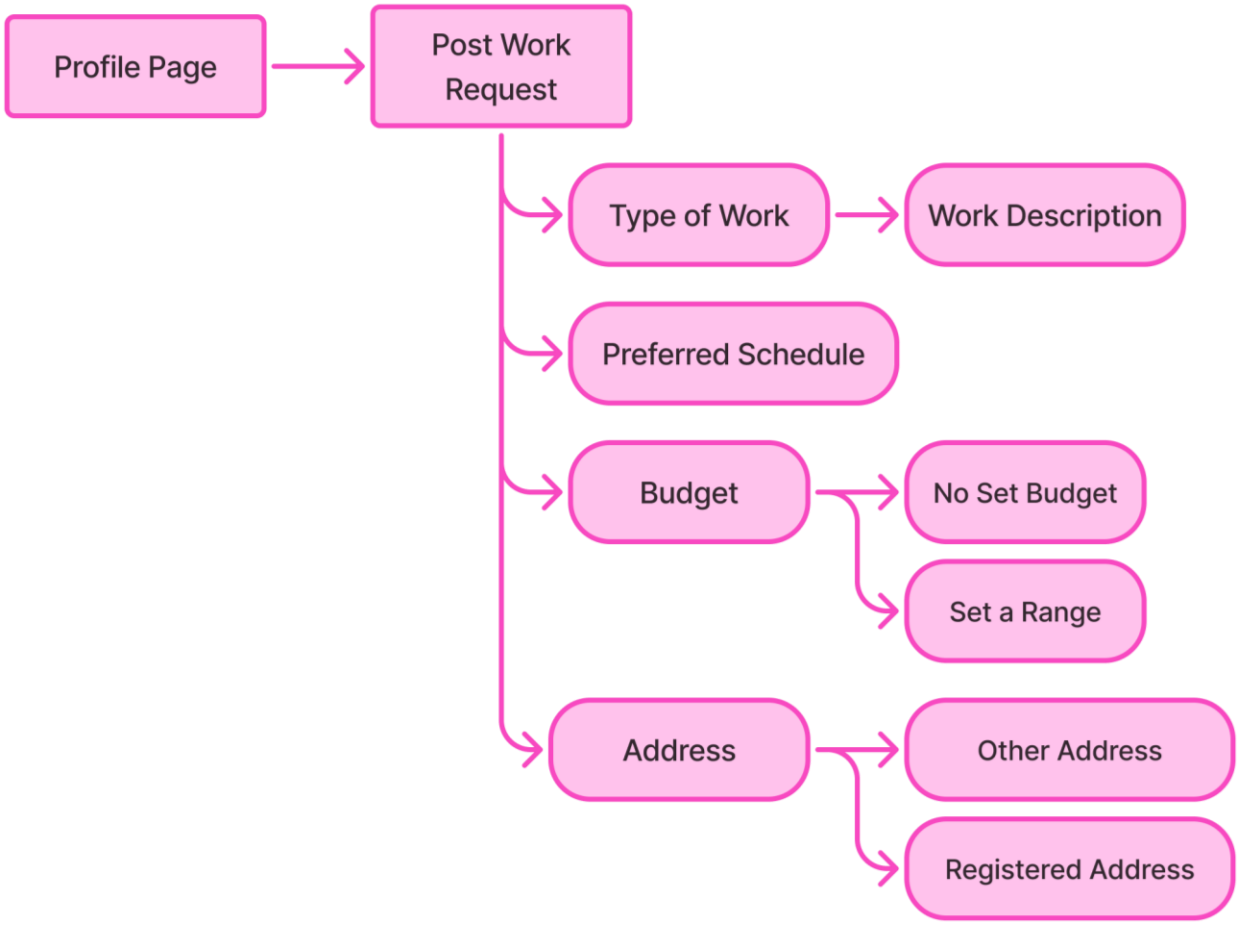


Figure 24: Use Case 2 Flowchart

This use case enables an authenticated user to create and publish a work request on the platform. The user specifies relevant job details such as type of work, description, preferred schedule, budget range, and location. Once submitted, the work request is stored in the system and becomes visible to eligible workers for discovery and application. This is a high priority use case, as it directly facilitates the core objective of connecting service seekers with workers. Successful execution of this use case results in an active work request that can receive applications or referrals until it is either cancelled or fulfilled.

3.1.3 Use Case 3: Searching and Filtering Work Requests

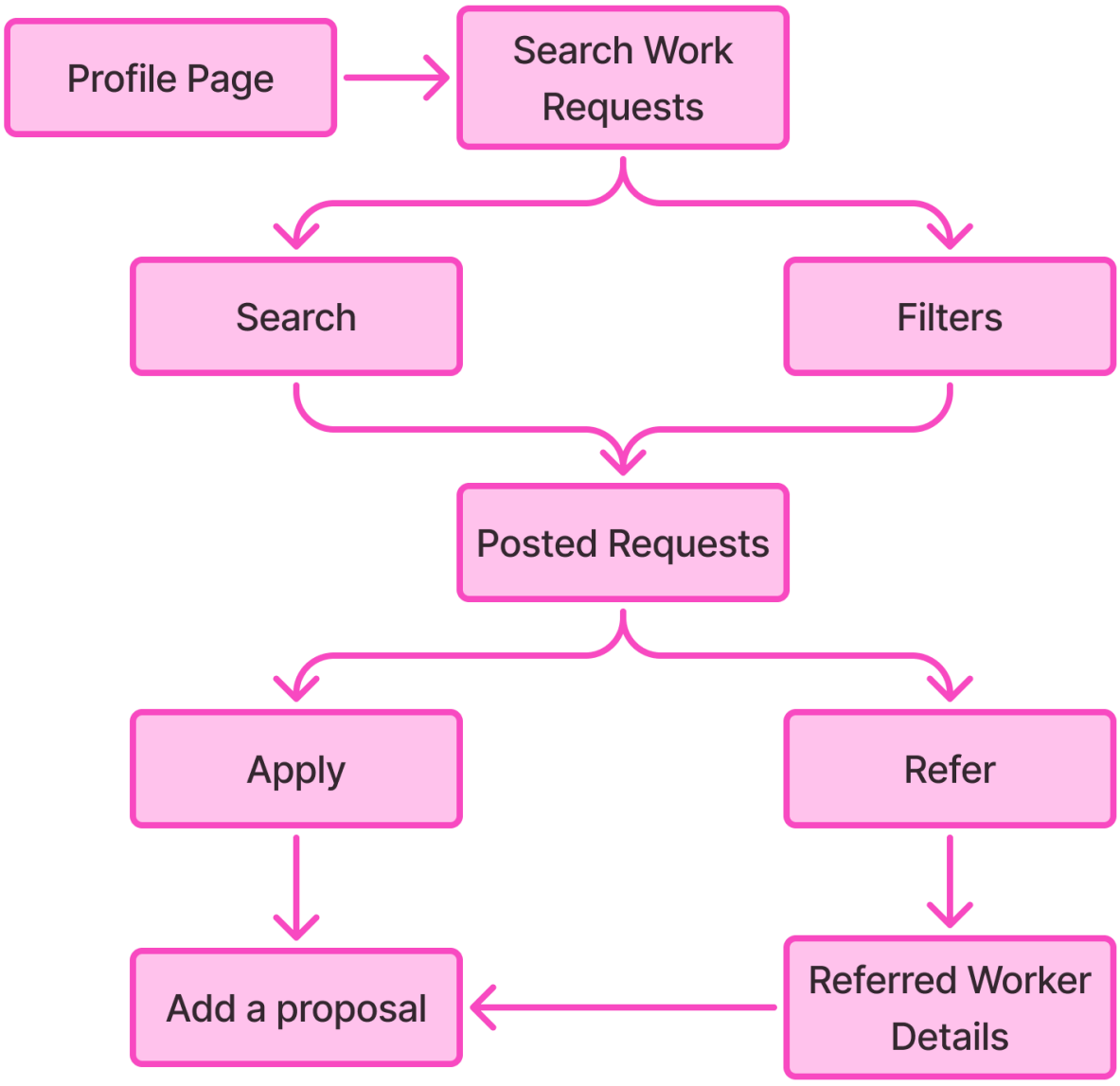


Figure 25: Use Case 3 Flowchart

This use case allows workers to efficiently discover suitable work opportunities available on the platform. An authenticated worker can browse all active work requests and apply search and filtering criteria such as job category, location, time preference, and budget range. The system processes these filters and presents a refined list of matching work requests. This use case improves usability and efficiency by reducing search overhead and ensuring relevance of displayed opportunities. Upon completion, the worker is presented with a filtered set of work requests aligned with their skills and availability.

3.1.4 Use Case 4: Applying, Referring, Negotiating, and Selecting an Applicant

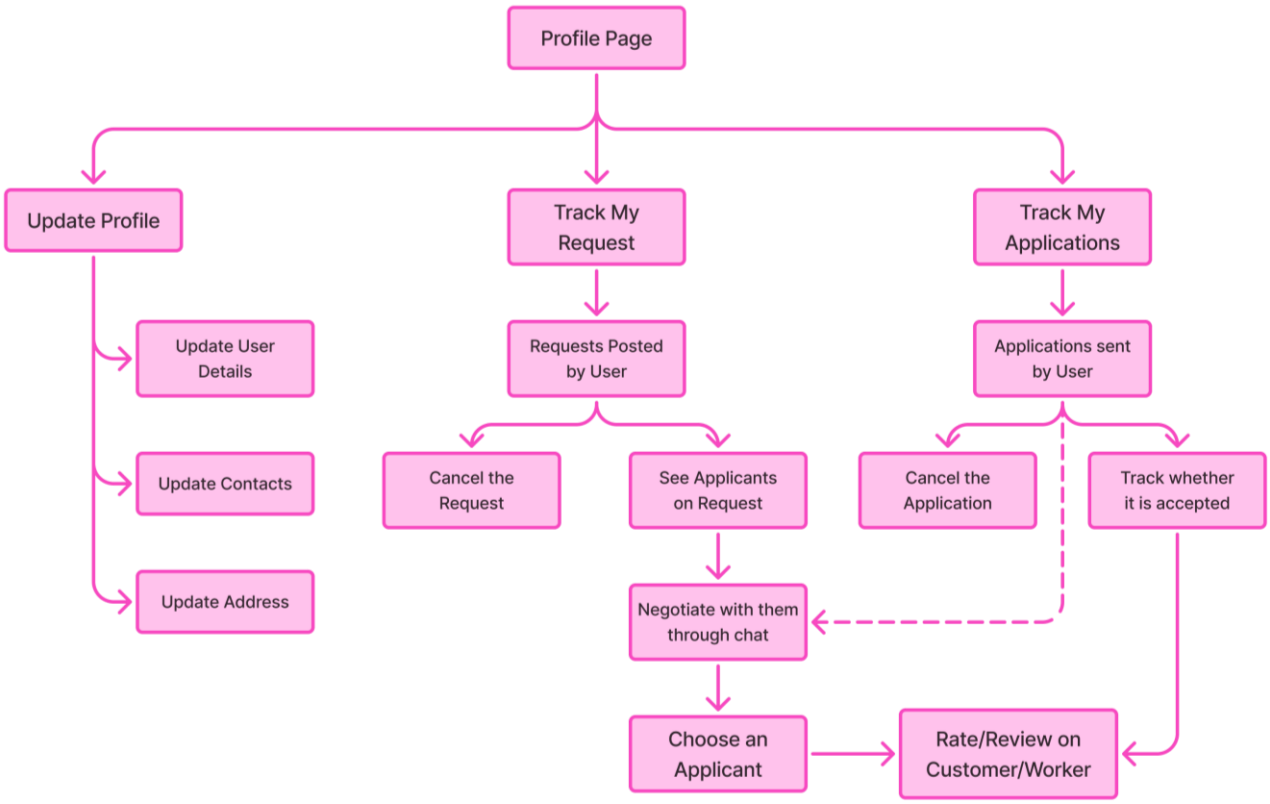


Figure 26: Use Case 4 Flowchart

This use case governs the interaction between users and workers during the job selection process. Workers can apply to a work request by submitting a proposal, or users and workers may refer an unregistered worker through an OTP-verified referral mechanism. The requesting user can review received applications, view worker profiles, and initiate temporary one-to-one chats to negotiate job details such as timing and pay. Once a suitable applicant is selected, the system confirms the application, automatically rejects all others, and removes all temporary chats related to that work request. This high priority use case ensures fair selection, controlled communication, and consistent state transitions within the system.

3.2 Class Diagrams

3.2.1 Class Diagram

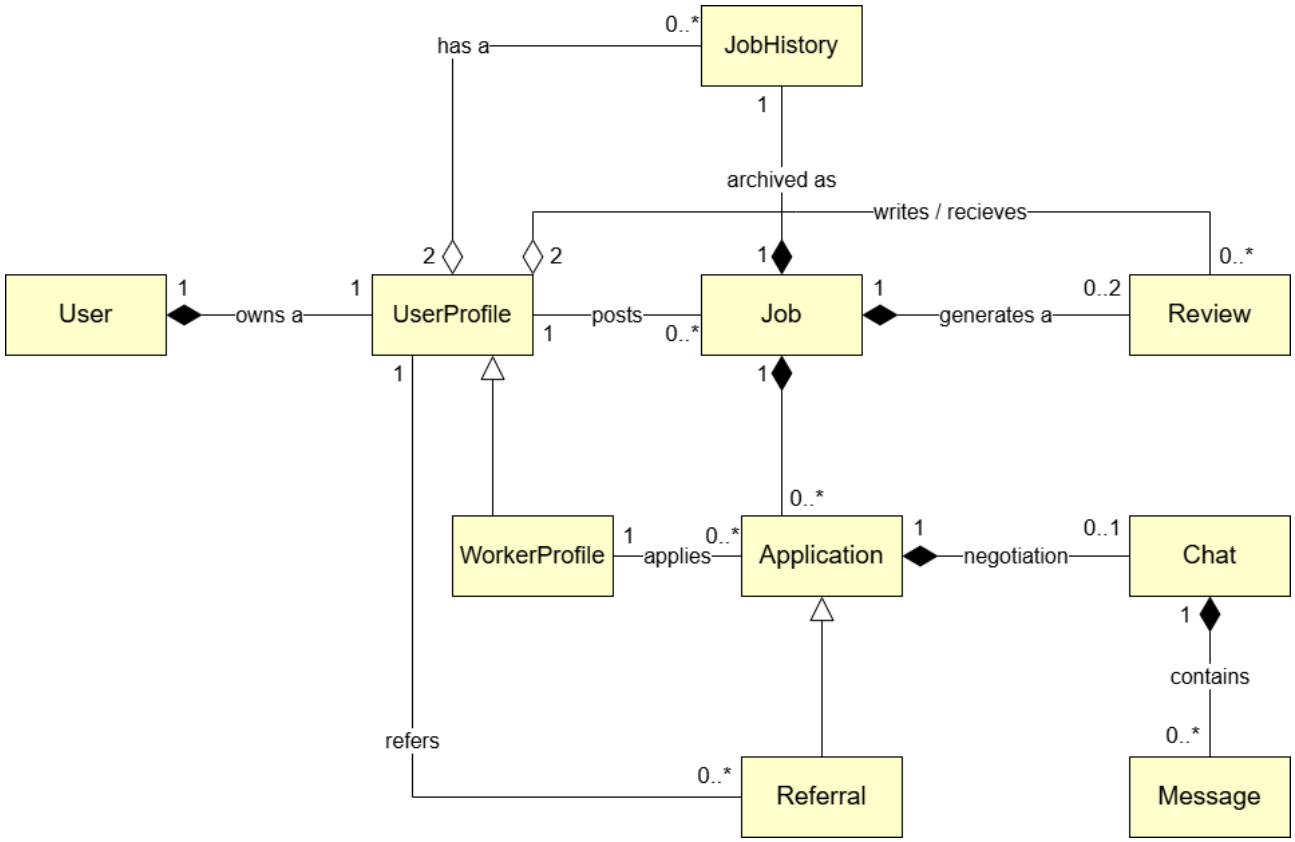
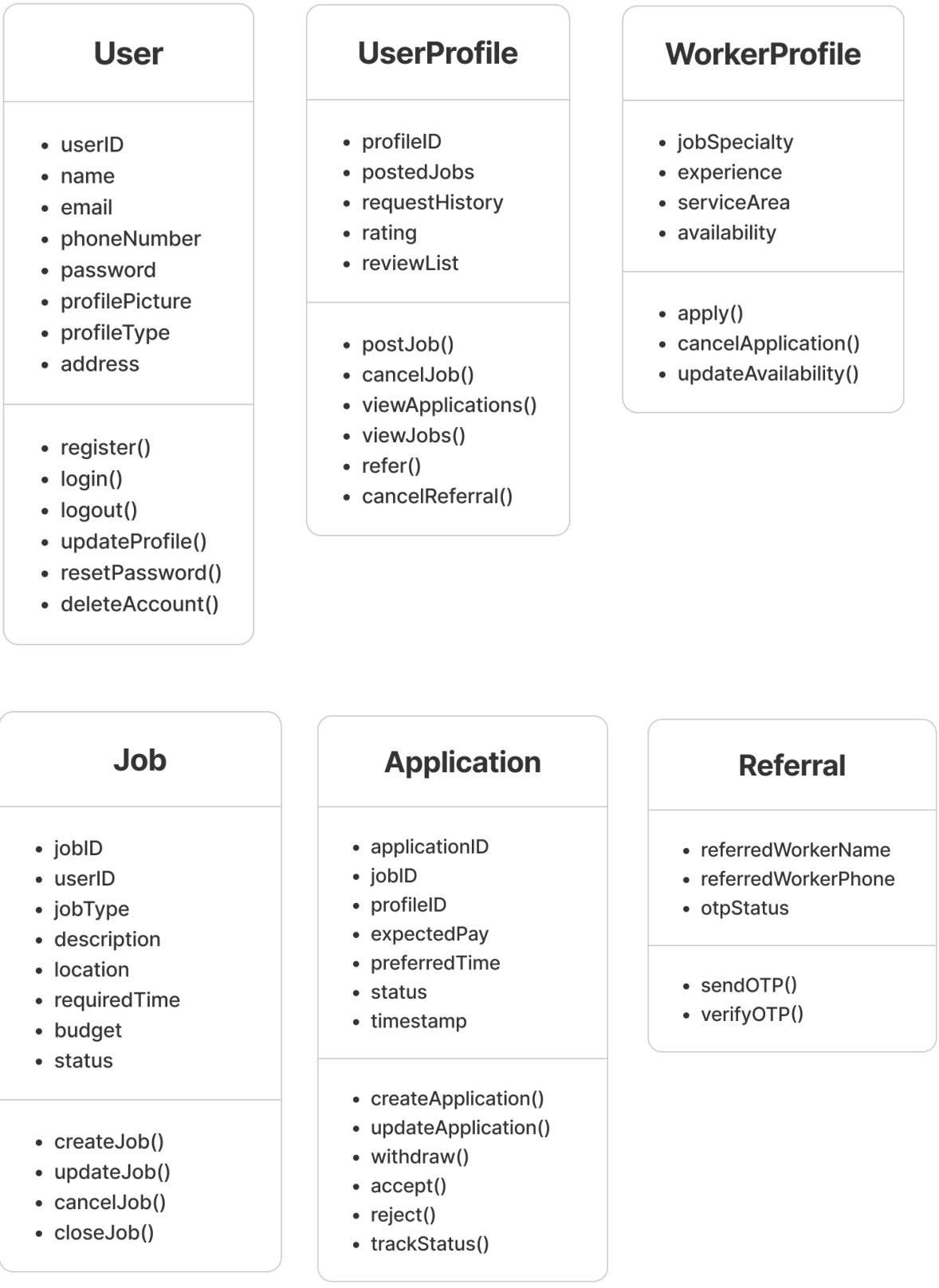
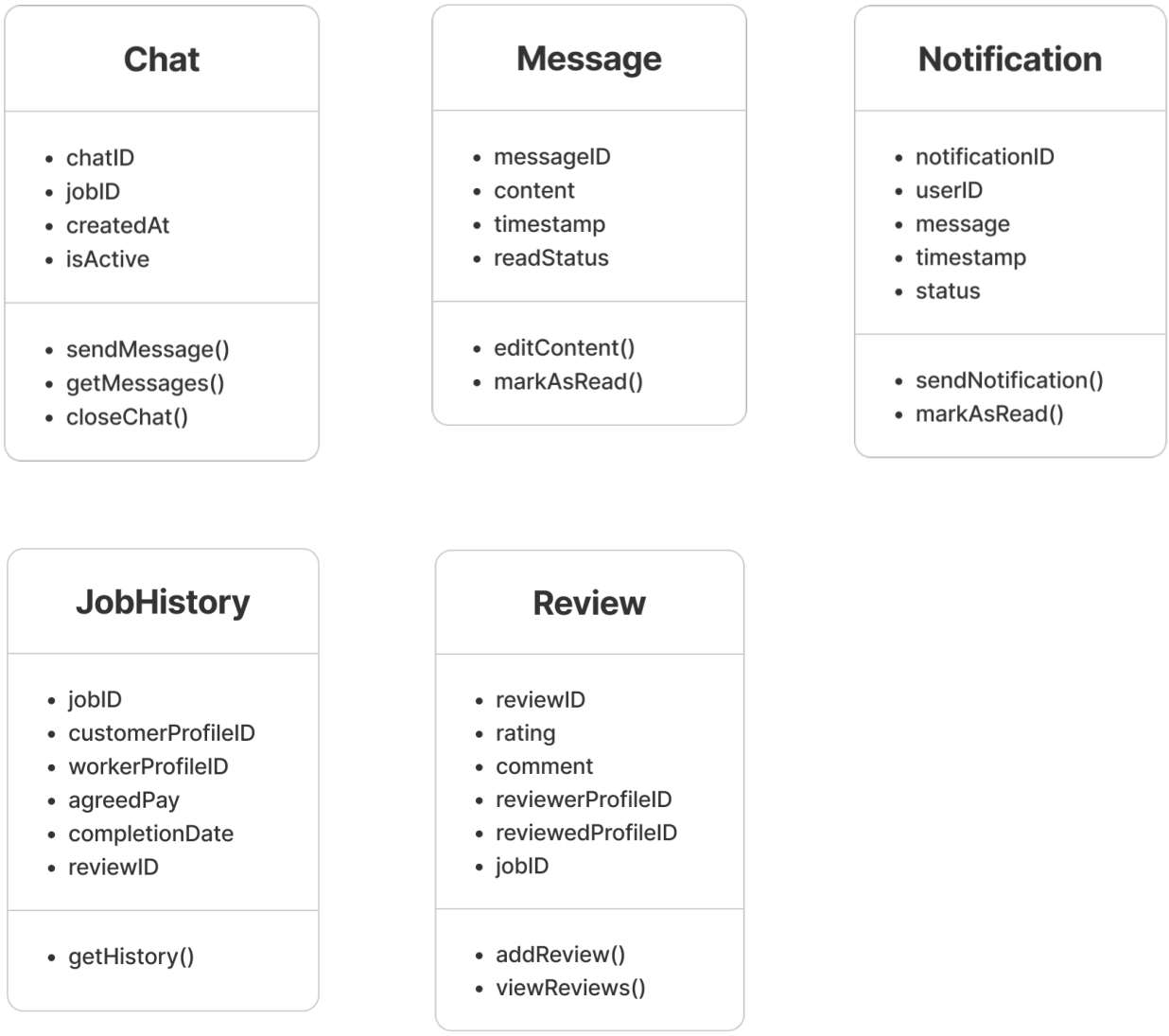


Figure 27: Class Diagram for KaamSetu

3.2.2 Classes


User
userID
name
email
phoneNumber
password
profilePicture
profileType
address
register()
login()
logout()
updateProfile()
resetPassword()
deleteAccount()
UserProfile
profileID
postedJobs
requestHistory
rating
reviewList
postJob()
cancelJob()
viewApplications()
viewJobs()
refer()
cancelReferral()
WorkerProfile
jobSpecialty
experience
serviceArea
availability
apply()
cancelApplication()
updateAvailability()
Job
jobID
userID
jobType
description
location
requiredTime
budget
status
createJob()
updateJob()
cancelJob()
closeJob()
Application
applicationID
jobID
profileID
expectedPay
preferredTime
status
timestamp
createApplication()
updateApplication()
withdraw()
accept()
reject()
trackStatus()
Referral
referredWorkerName
referredWorkerPhone
otpStatus
sendOTP()
verifyOTP()


Chat
chatID
jobID
createdAt
isActive
sendMessage()
getMessages()
closeChat()
Message
messageID
content
timestamp
readStatus
editContent()
markAsRead()
Notification
notificationID
userID
message
timestamp
status
sendNotification()
markAsRead()
JobHistory
jobID
customerProfileID
workerProfileID
agreedPay
completionDate
reviewID
getHistory()
Review
reviewID
rating
comment
reviewerProfileID
reviewedProfileID
jobID
addReview()
viewReviews()

3.2.3 Classes Descriptions

- ***User Class***
  - *Attributes*
    - **userID** – Unique identifier for the user account.
    - **name** – Full name of the user.
    - **email** – Registered email address used for authentication and communication.
    - **phoneNumber** – Contact number of the user.
    - **password** – Encrypted password for user authentication.
    - **profilePicture** – Display image associated with the user account.
    - **profileType** – Indicates whether the user is a worker or non-worker.
    - **address** – Residential or service-related address of the user.
  - *Methods*
    - **register()** – Creates a new user account in the system.
    - **login()** – Authenticates the user and starts a session.
    - **logout()** – Ends the user's active session.
    - **updateProfile()** – Updates personal and account details of the user.
    - **resetPassword()** – Allows the user to reset their password securely.
    - **deleteAccount()** – Permanently removes the user account from the system.

- ***UserProfile Class***
  - *Attributes*
    - **profileID** – Unique identifier for the user profile.
    - **postedJobs** – Collection of jobs posted by the user.
    - **requestHistory** – Record of completed job engagements associated with the user.
    - **rating** – Aggregated rating score received by the user.
    - **reviewList** – List of reviews written for or by the user.
  - *Methods*
    - **postJob()** – Creates and publishes a new job request.
    - **cancelJob()** – Cancels a job request before completion.
    - **viewApplications()** – Displays applications received for posted jobs.
    - **viewJobs()** – Shows available jobs for browsing or reference.
    - **refer()** – Refers an unregistered worker to a job.
    - **cancelReferral()** – Withdraws a previously made referral.

- ***WorkerProfile Class***
  - *Attributes*
    - **jobSpecialty** – Type of work or skill offered by the worker.
    - **experience** – Description of the worker's prior work experience.
    - **serviceArea** – Geographic area where the worker provides services.
    - **availability** – Time slots during which the worker is available.
  - *Methods*
    - **apply()** – Applies to a job by creating an application.
    - **cancelApplication()** – Withdraws an existing job application.
    - **updateAvailability()** – Updates the worker's available working times.

- ***Job Class***
  - *Attributes*
    - **jobID** – Unique identifier for the job.
    - **userID** – Identifier of the user who posted the job.
    - **jobType** – Category or type of work requested.
    - **description** – Detailed explanation of the job requirements.
    - **location** – Location where the job is to be performed.
    - **requiredTime** – Expected time or schedule for the job.
    - **budget** – Payment range or agreed cost for the job.
    - **status** – Current state of the job (open, cancelled, completed).
  - *Methods*
    - **createJob()** – Initializes and posts a new job.
    - **updateJob()** – Modifies job details while the job is active.
    - **cancelJob()** – Cancels the job before assignment or completion.
    - **closeJob()** – Marks the job as completed after execution.

- ***Application Class***
  - *Attributes*
    - **applicationID** – Unique identifier for the job application.
    - **jobID** – Identifier of the job being applied to.
    - **profileID** – Identifier of the applicant's profile.
    - **expectedPay** – Payment expected by the applicant.
    - **preferredTime** – Preferred working time proposed by the applicant.
    - **status** – Current state of the application (pending, accepted, rejected).
    - **timestamp** – Time when the application was created.
  - *Methods*
    - **createApplication()** – Submits a new application for a job.
    - **updateApplication()** – Updates application details.
    - **withdraw()** – Withdraws the application before acceptance.
    - **accept()** – Accepts the application for the job.
    - **reject()** – Rejects the application.
    - **trackStatus()** – Retrieves the current application status.

- ***Referral Class***
  - *Attributes*
    - **referredWorkerName** – Name of the worker being referred.
    - **referredWorkerPhone** – Contact number of the referred worker.
    - **otpStatus** – Status indicating whether OTP verification is complete.
  - *Methods*
    - **sendOTP()** – Sends OTP to the referred worker for verification.
    - **verifyOTP()** – Verifies the OTP submitted by the referred worker.

- ***Chat Class***
  - *Attributes*
    - **chatID** – Unique identifier for the chat session.
    - **jobID** – Identifier of the job associated with the chat.
    - **createdAt** – Timestamp when the chat was initiated.
    - **isActive** – Indicates whether the chat session is currently active.
  - *Methods*
    - **sendMessage()** – Sends a message within the chat.
    - **getMessages()** – Retrieves all messages in the chat.
    - **closeChat()** – Terminates the chat session.

- ***Message Class***
  - *Attributes*
    - **messageID** – Unique identifier for the message.
    - **content** – Text content of the message.
    - **timestamp** – Time when the message was sent.
    - **readStatus** – Indicates whether the message has been read.
  - *Methods*
    - **editContent()** – Edits the content of an existing message.
    - **markAsRead()** – Marks the message as read.

- ***Notification Class***
  - *Attributes*
    - **notificationID** – Unique identifier for the notification.
    - **userID** – Identifier of the user receiving the notification.
    - **message** – Notification message content.
    - **timestamp** – Time when the notification was generated.
    - **status** – Indicates whether the notification has been read.
  - *Methods*
    - **sendNotification()** – Sends a notification to a user.
    - **markAsRead()** – Marks the notification as read.

- ***JobHistory Class***
  - *Attributes*
    - **jobID** – Identifier of the completed job.
    - **customerProfileID** – Profile ID of the job poster.
    - **workerProfileID** – Profile ID of the worker who completed the job.
    - **agreedPay** – Final agreed payment for the job.
    - **completionDate** – Date when the job was completed.
    - **reviewID** – Identifier of the associated review.
  - *Methods*
    - **getHistory()** – Retrieves the job's historical record.

- ***Review Class***
  - *Attributes*
    - **reviewID** – Unique identifier for the review.
    - **rating** – Numerical rating given after job completion.
    - **comment** – Textual feedback provided in the review.
    - **reviewerProfileID** – Profile ID of the user who wrote the review.
    - **reviewedProfileID** – Profile ID of the user being reviewed.
    - **jobID** – Identifier of the job associated with the review.
  - *Methods*
    - **addReview()** – Creates and stores a new review.
    - **viewReviews()** – Displays reviews related to a user or job.

3.3 Sequence Diagrams

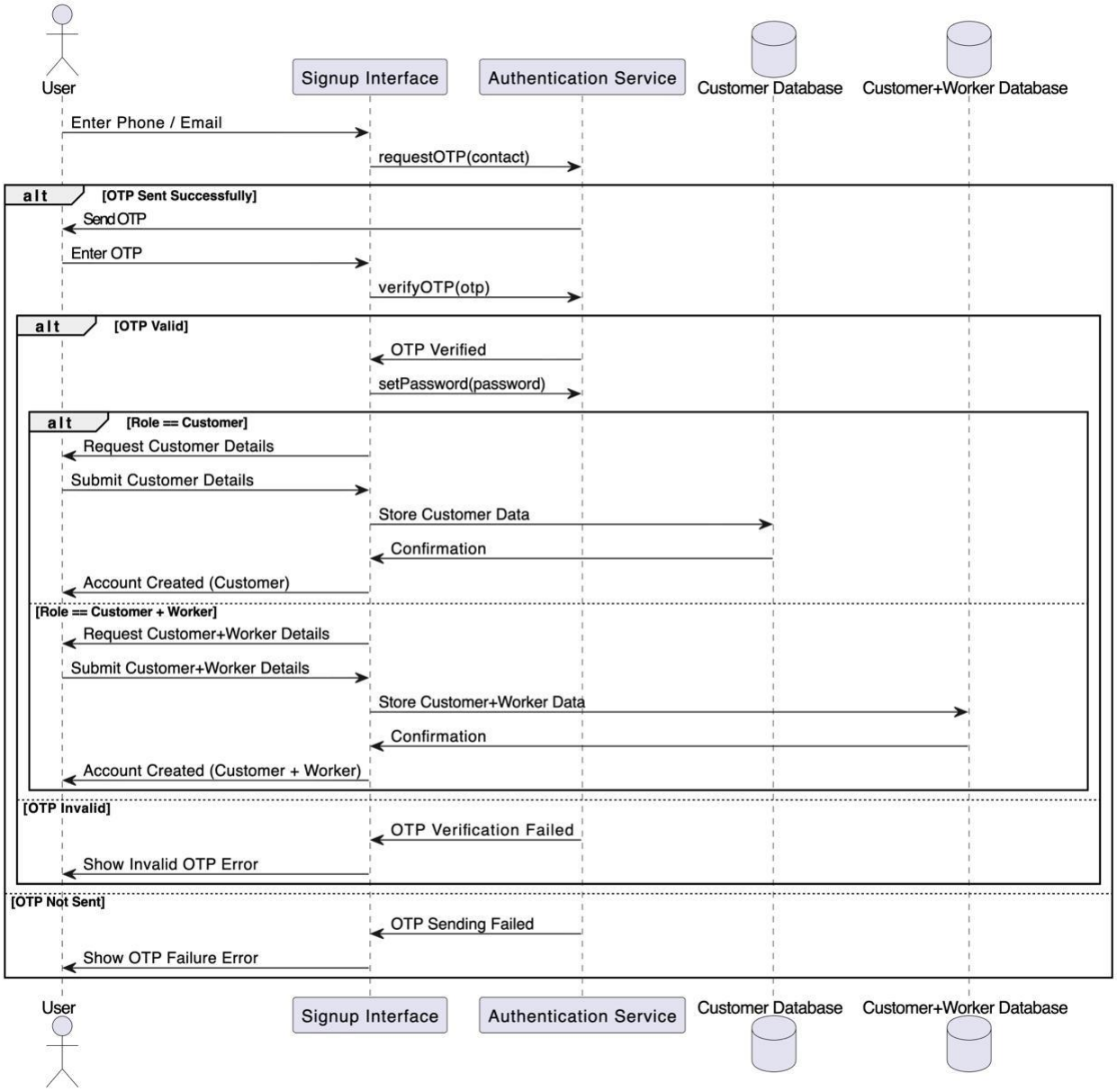


Figure 28: Signup Page Sequence Diagram

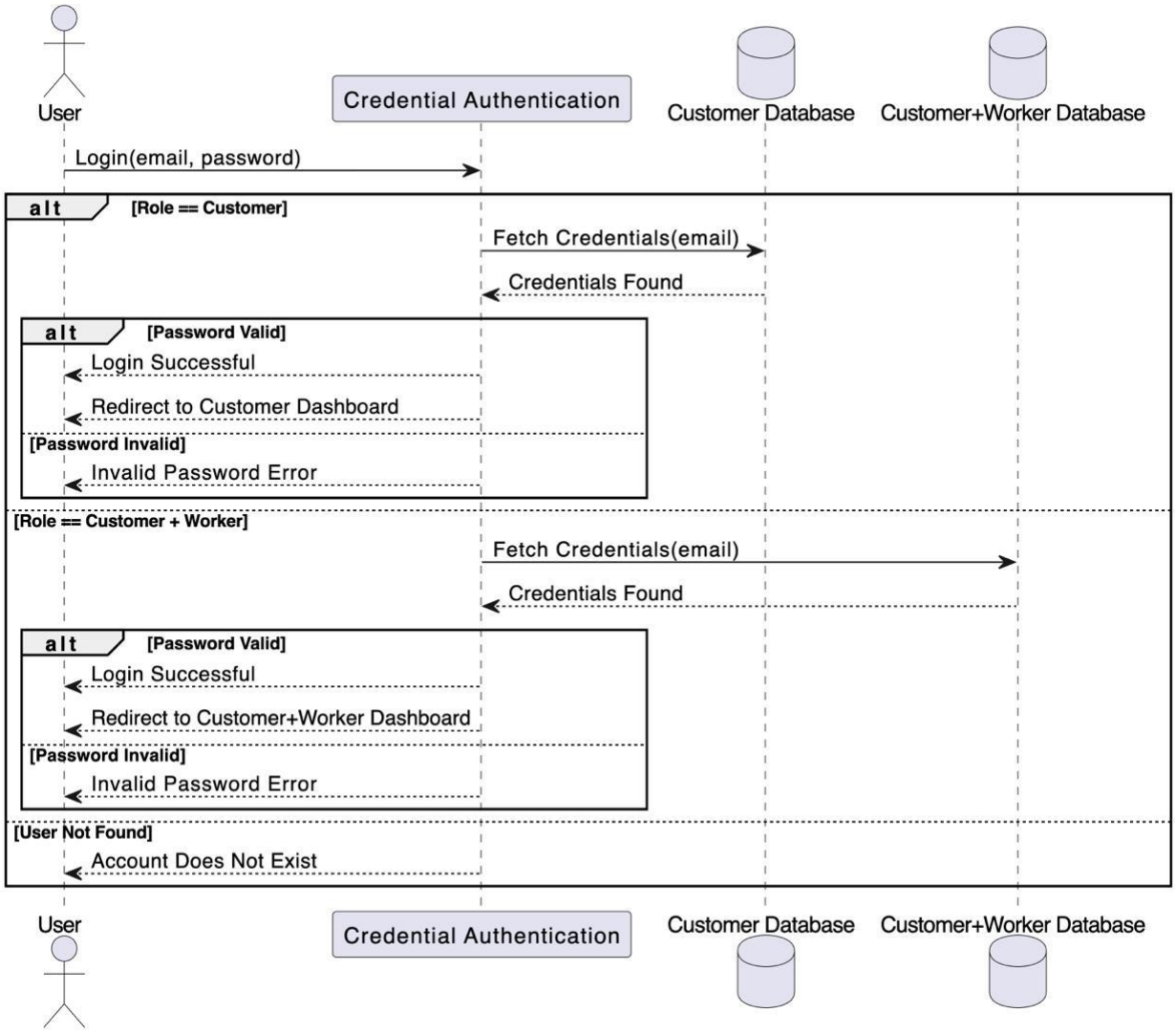


Figure 29: Login Page Sequence Diagram

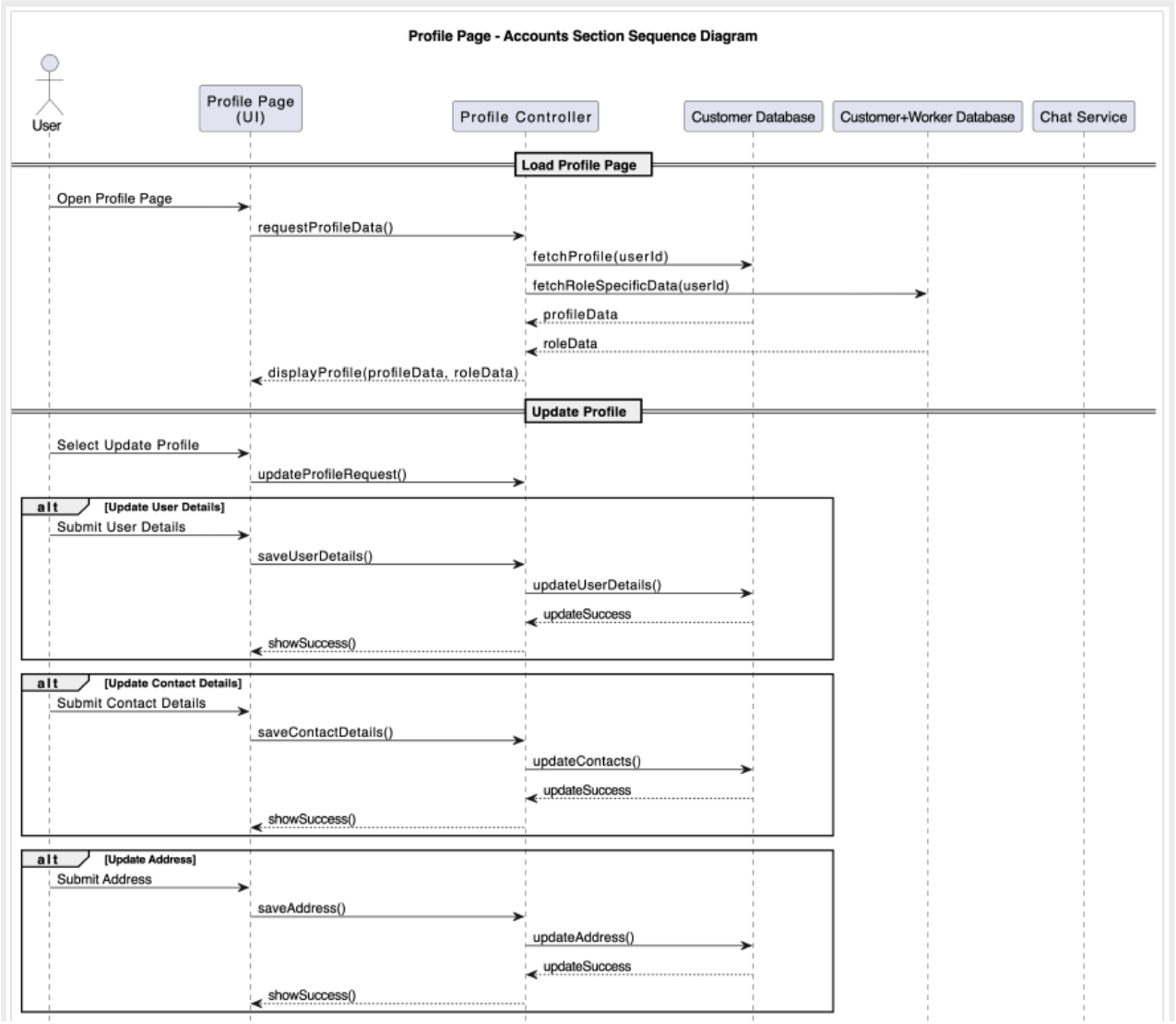


Figure 30: Account Section Sequence Diagram - 1

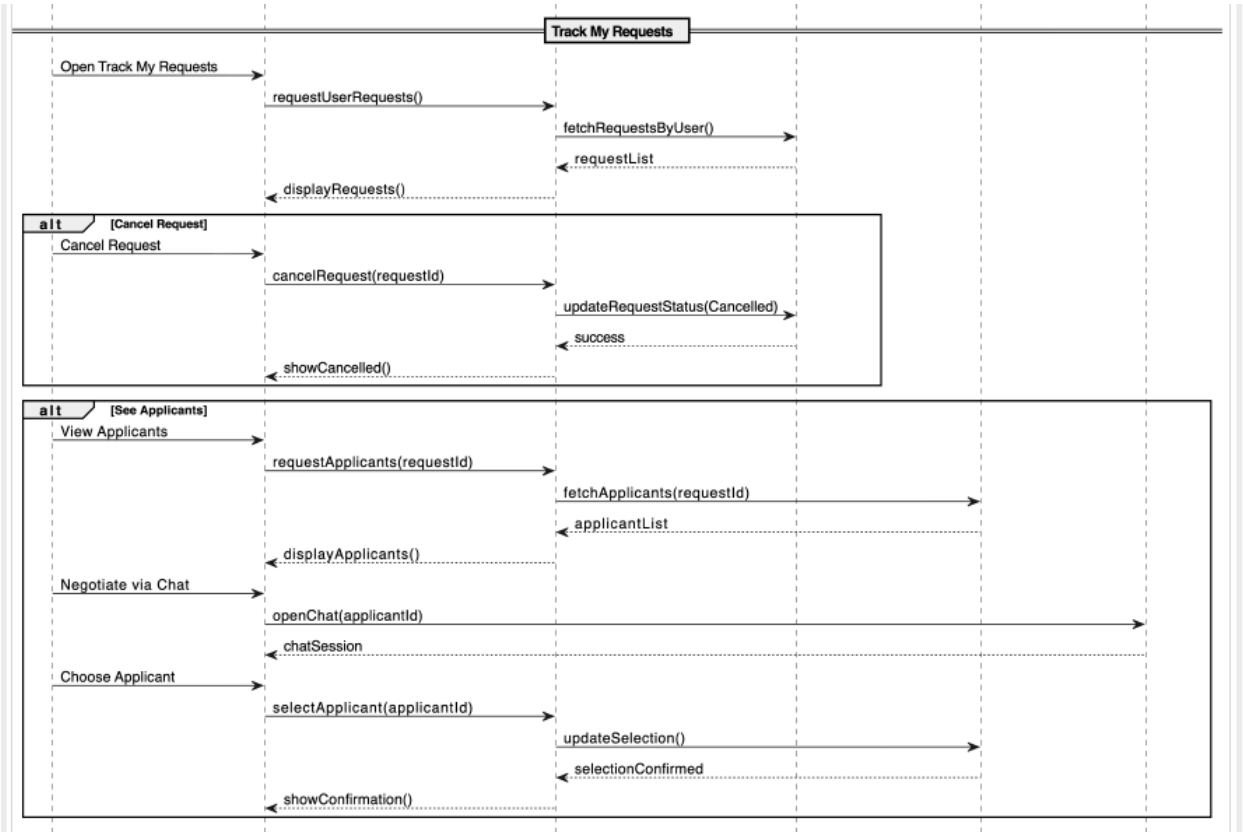


Figure 31: Account Section Sequence Diagram – 2

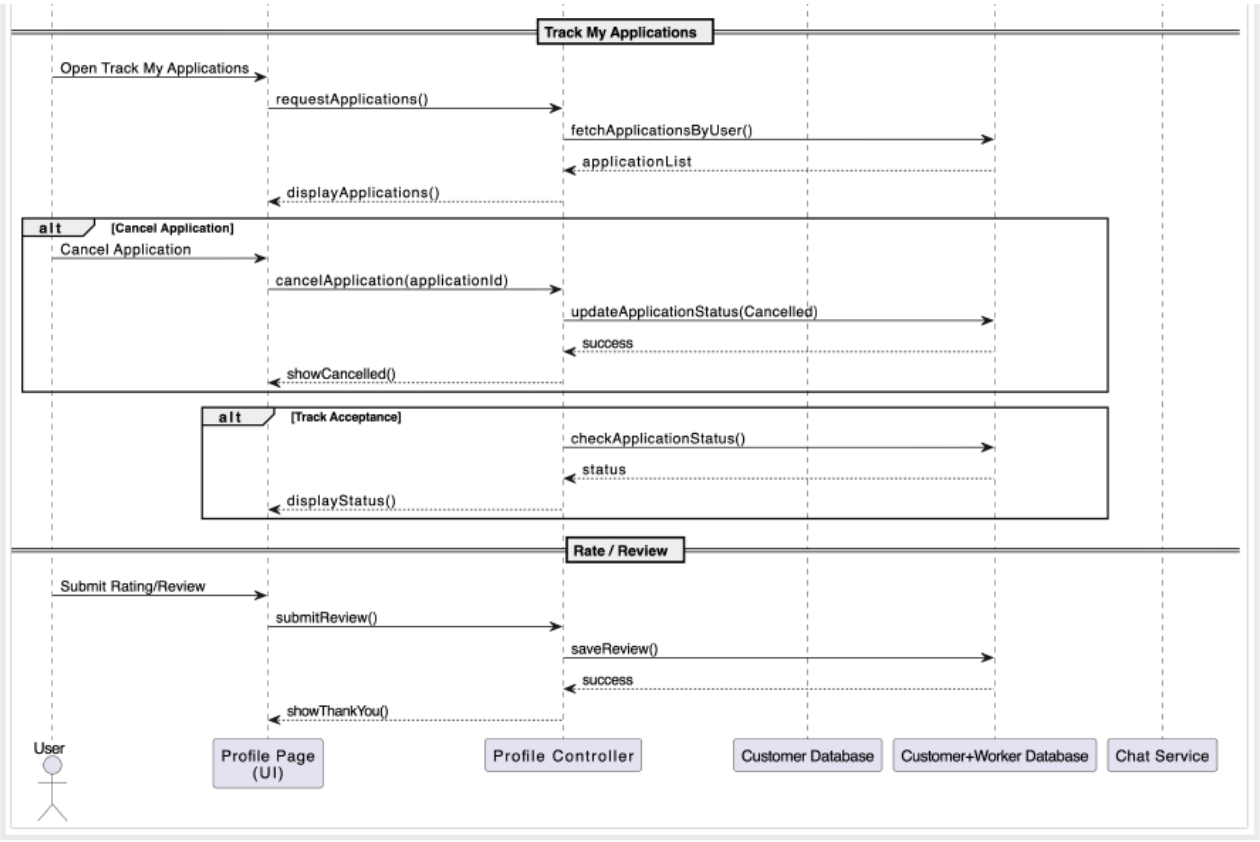


Figure 32: Account Section Sequence Diagram – 3

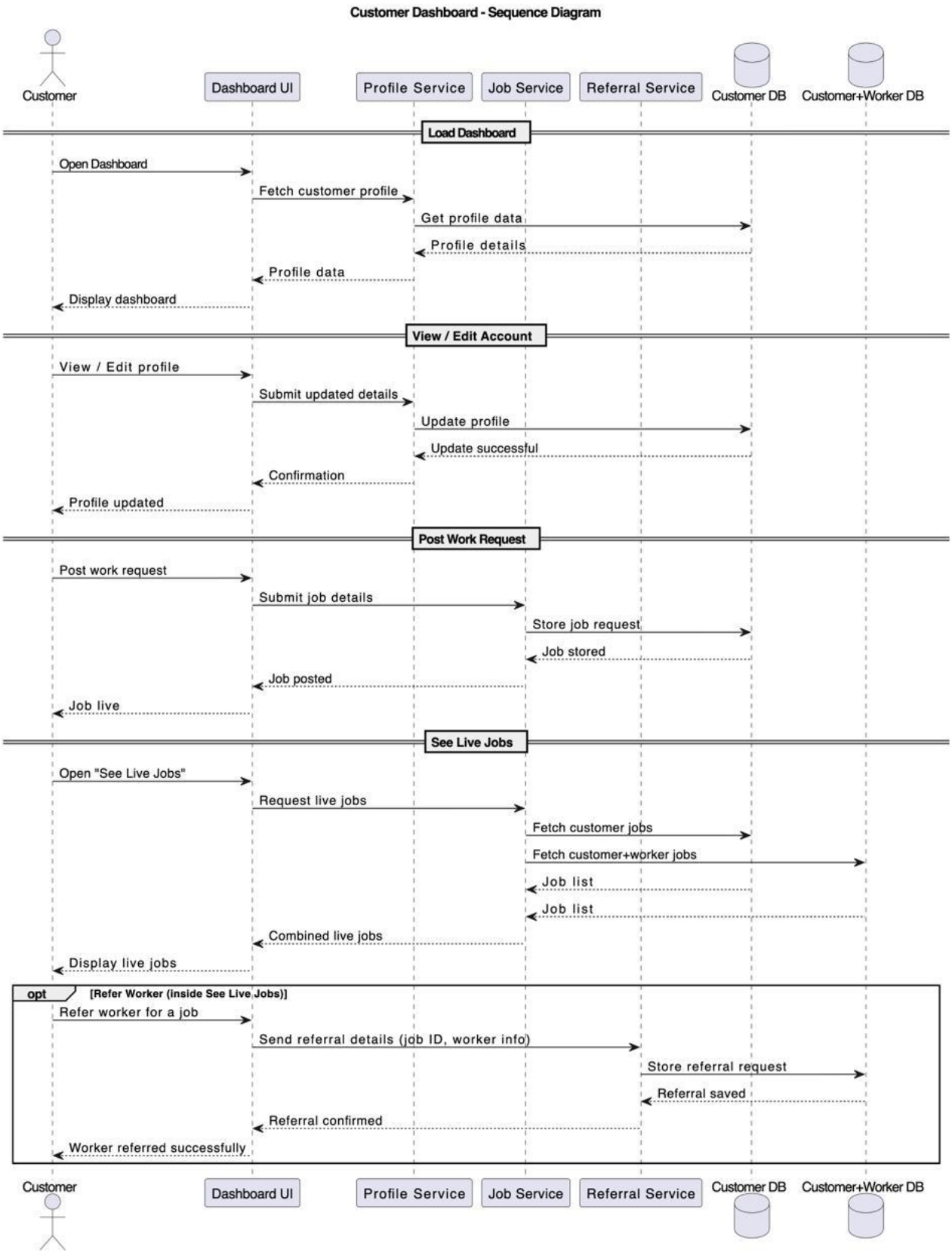


Figure 33: Non-Worker Dashboard Sequence Diagram

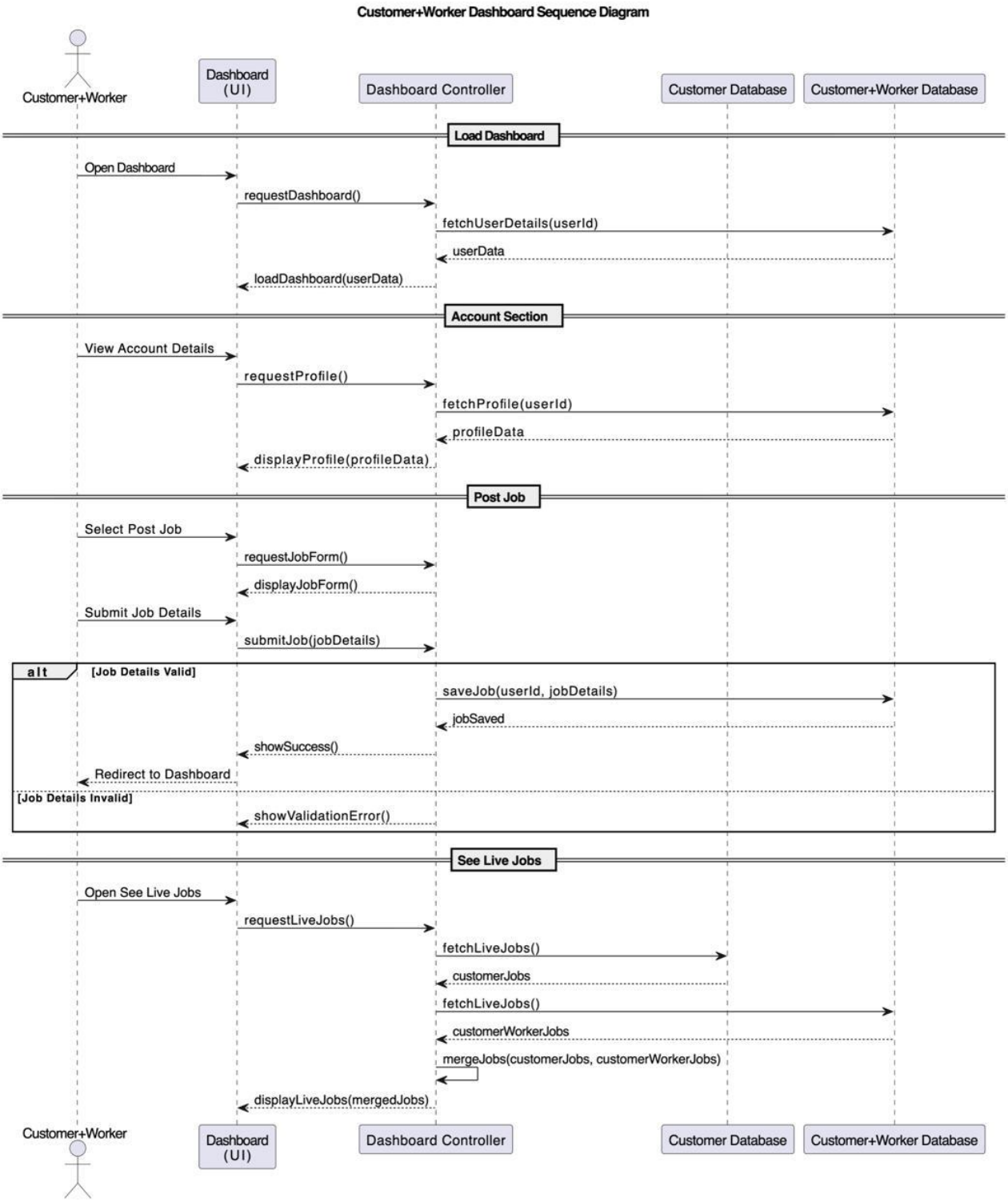


Figure 34: Worker Dashboard Sequence Diagram

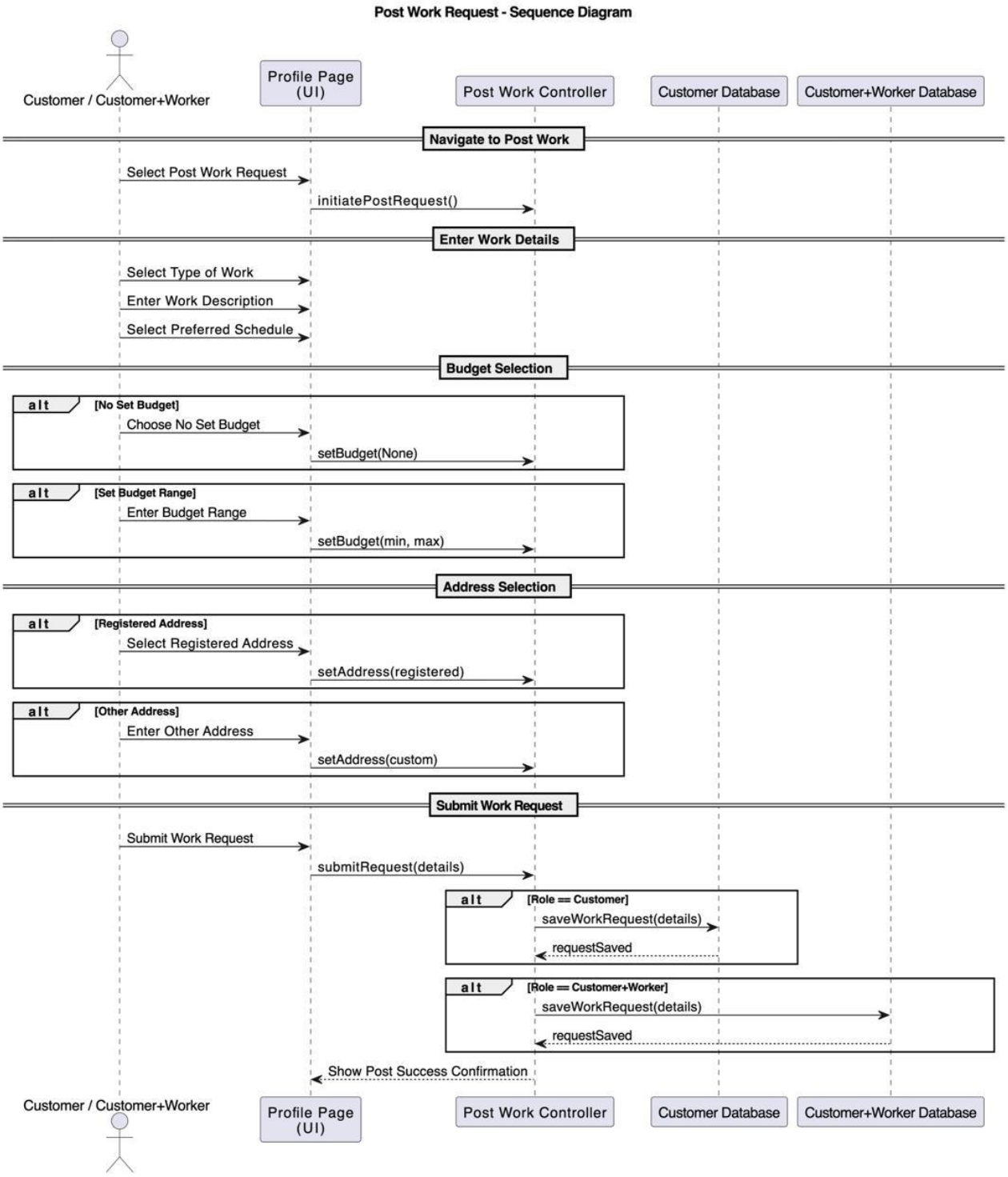


Figure 35: Post Work Request Sequence Diagram

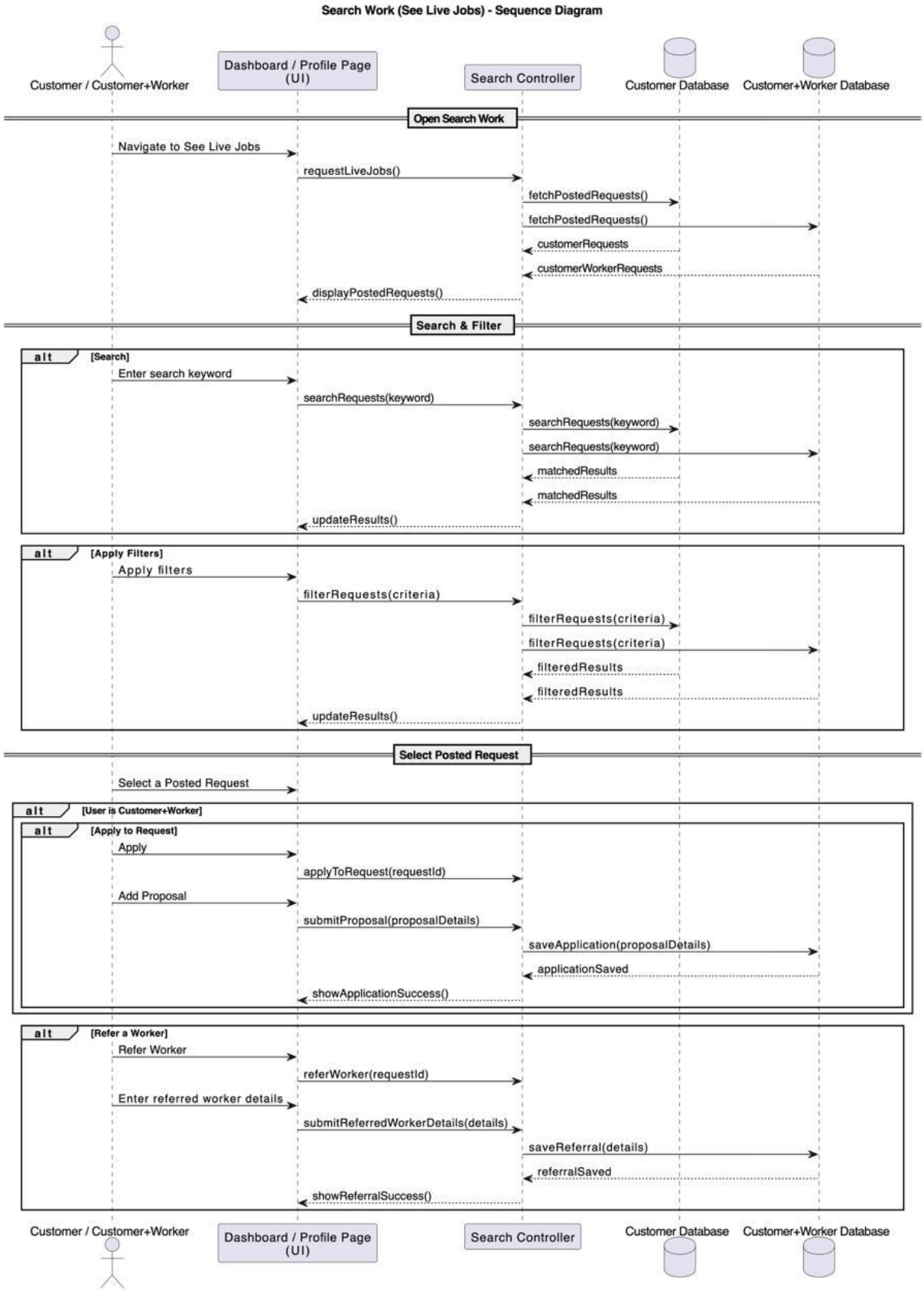


Figure 36: Search Work Sequence Diagram

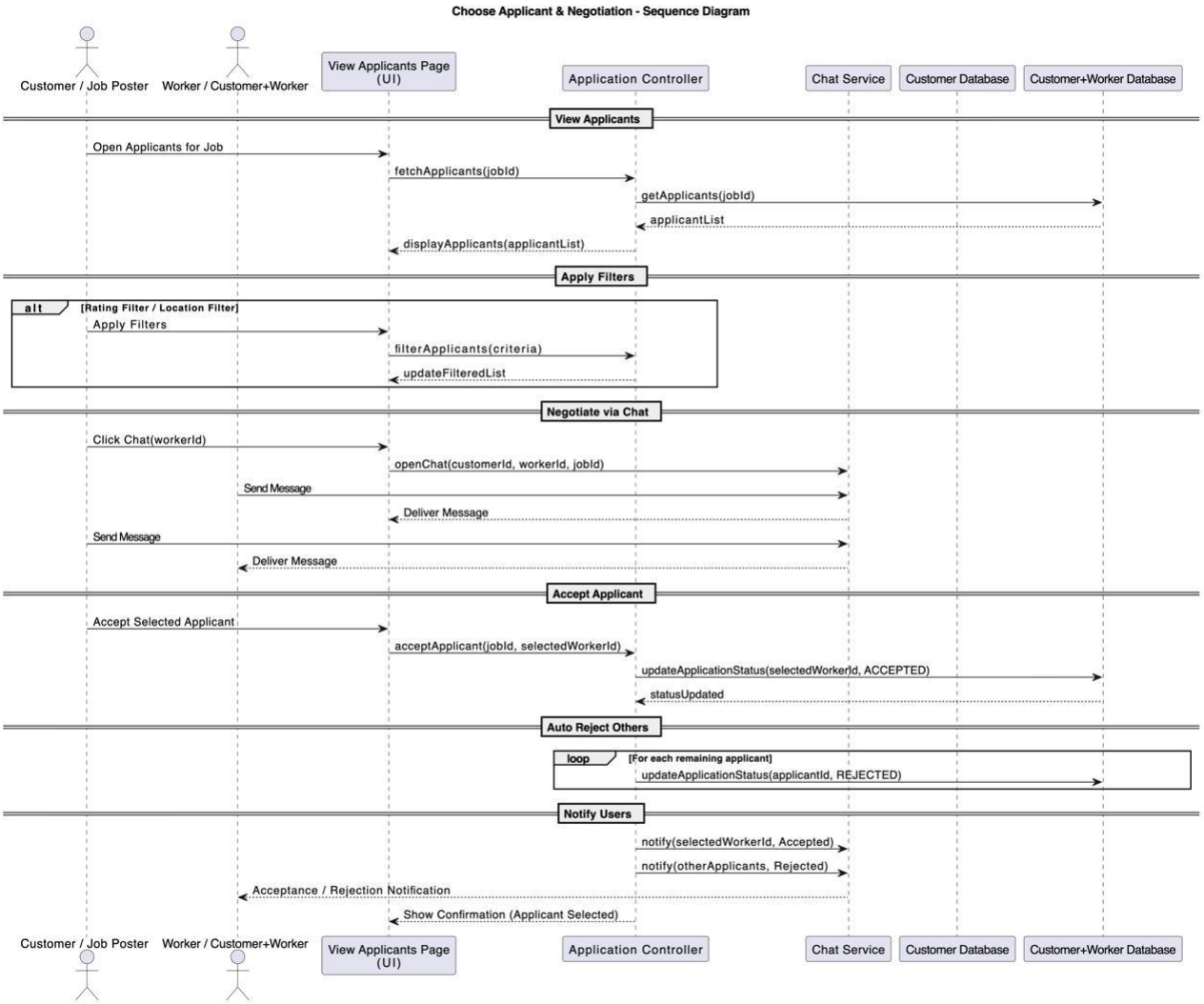


Figure 37: Choose Applicant & Negotiation Sequence Diagram

3.4 State Diagrams

The application's response to each request can be illustrated through a state diagram. The system can transition between different states based on user inputs and system actions as described in the state diagram below:

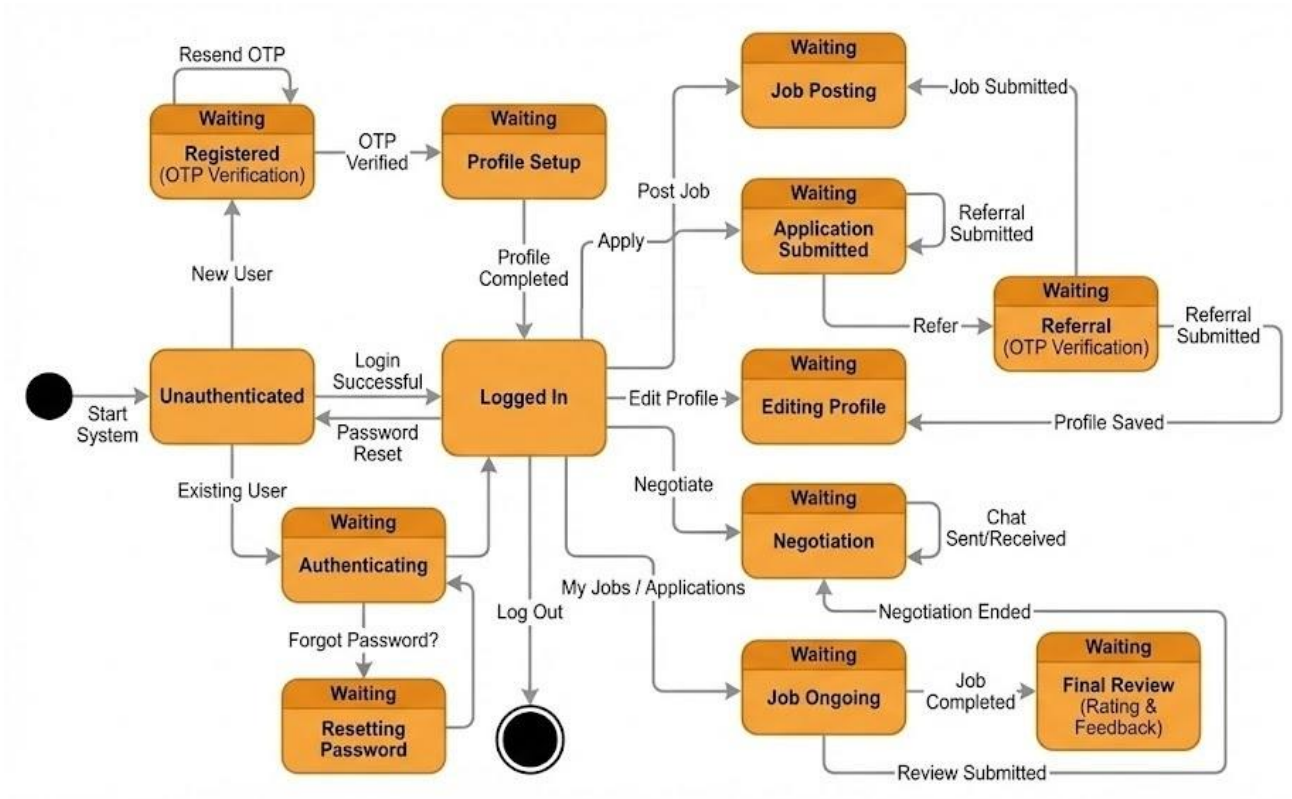
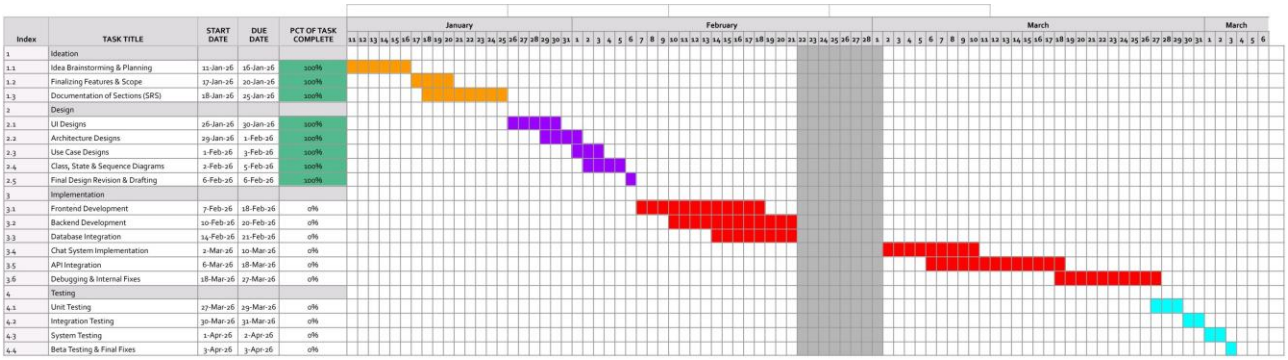


Figure 38: State Diagram for KaamSetu

4 Project Plan

4.1 Project KaamSetu Timeline

START DATE: 13 – Jan - 2026



Index	TASK TITLE	START DATE	DUE DATE	PCT OF TASK COMPLETE
1	Ideation			
1.1	Idea Brainstorming & Planning	11-Jan-26	16-Jan-26	100%
1.2	Finalizing Features & Scope	17-Jan-26	20-Jan-26	100%
1.3	Documentation of Sections (SRS)	18-Jan-26	25-Jan-26	100%
2	Design			
2.1	UI Designs	26-Jan-26	30-Jan-26	100%
2.2	Architecture Designs	29-Jan-26	1-Feb-26	100%
2.3	Use Case Designs	1-Feb-26	3-Feb-26	100%
2.4	Class, State & Sequence Diagrams	2-Feb-26	5-Feb-26	100%
2.5	Final Design Revision & Drafting	6-Feb-26	6-Feb-26	100%
3	Implementation			
3.1	Frontend Development	7-Feb-26	18-Feb-26	0%
3.2	Backend Development	10-Feb-26	20-Feb-26	0%
3.3	Database Integration	14-Feb-26	21-Feb-26	0%
3.4	Chat System Implementation	2-Mar-26	10-Mar-26	0%
3.5	API Integration	6-Mar-26	18-Mar-26	0%
3.6	Debugging & Internal Fixes	18-Mar-26	27-Mar-26	0%
4	Testing			
4.1	Unit Testing	27-Mar-26	29-Mar-26	0%
4.2	Integration Testing	30-Mar-26	31-Mar-26	0%
4.3	System Testing	1-Apr-26	2-Apr-26	0%
4.4	Beta Testing & Final Fixes	3-Apr-26	3-Apr-26	0%

Figure 39: Gantt Chart covering the progress of *Null Pointers*

Appendix A - Group Log

Meeting Time	Agenda
26 Jan 2026	Initial team meeting to understand the problem statement and project objectives. Discussed the scope of KaamSetu and brainstormed core design elements.
29 Jan 2026	Started outlining the Context Design and Human Interface Design sections.
1 Feb 2026	Discussed system architecture and agreed on using the MVC design pattern. Began work on Architecture Design and Use Case Diagrams.
4 Feb 2026	Reviewed Object-Oriented Design components including class diagrams and use case flows. Assigned remaining documentation tasks among team members.
6 Feb 2026	Integrated all sections of the Software Design Document. Reviewed content for consistency, clarity, and completeness before final submission preparation.